

Detailed Proofs

1. Additional Definitions Required for Proofs

1.1. Error Correction Codes

One, can construct a zero-knowledge code using a randomized encoding function for an error-correcting code $\mathcal{C}$. **Randomized Encoding Function [1].** Let $\mathcal{C} : \Sigma^n \rightarrow \Sigma^{n'}$ be an error-correcting code. A randomized encoding function of $\mathcal{C}$, is a random map $Enc : \Sigma^m \rightarrow \text{img}(\mathcal{C})$ such that for $m < n$ and any messages $\mathbf{m} \neq \mathbf{m}' \in \Sigma^m$ the set of all images of $Enc(\mathbf{m})$ and $Enc(\mathbf{m}')$ are disjointed.

Given that, a zero-knowledge code is defined as follows:

Definition 1 (Zero Knowledge Code [1]). Let $\mathcal{C} : \Sigma^n \rightarrow \Sigma^{n'}$ be an error-correcting code and $Enc : \Sigma^m \rightarrow \Sigma^n$ a randomized encoding function of $\mathcal{C}$. We say that $\mathcal{C}$ is l -zero knowledge (with respect to Enc) if there exists a PPT simulator for any adversary that can make at most l queries over the codeword, and for any input $\mathbf{m} \in \Sigma^m$, the following views are identically distributed:

- 1) The view $View_A^R(\mathbf{m}, k, m)$ when the adversary has oracle access to $\mathbf{c} \leftarrow Enc(\mathbf{m})$.
- 2) The view $View_A^S(k, m)$ when the adversary's queries are replied by simulator $\mathcal{S}$.

Finally, we provide the following lemma that will be used when analyzing the soundness of our scheme:

Lemma 1 (Ames et al. [2], Roth and Zemor). Let $e < d/3$ where d the minimum distance of a code $\mathcal{C}$ over a finite field $\mathbb{F}$. If $\text{Dist}(\mathcal{U}, \mathcal{C}^m) > e$, then for a random $\mathbf{u}$ in the row-span of $\mathcal{U}$, it holds that $\Pr[\text{Dist}(\mathbf{u}, \mathcal{C}) < e] \leq (e + 1)/|\mathbb{F}|$.

1.2. Interactive Proofs of Proximity

Definition 2. Let $IOP = (\mathcal{P}, \mathcal{V})$ be a k -round IOP protocol for the relation $\mathcal{R}$. IOP is complete if for every $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$ it holds that:

$$\Pr \left(\begin{array}{l} \mathcal{V}^{(\pi^{(0)}, \dots, \pi^{(k)})}(r_1, \dots, r_k) = 1, \\ \pi^{(0)} \leftarrow \mathcal{P}(\mathbf{x}, \mathbf{w}) \\ \dots \\ \pi^{(k)} \leftarrow \mathcal{P}(\mathbf{x}, \mathbf{w}, r_1, \dots, r_k) \end{array} \right) = 1$$

An IOP is sound with soundness error ϵ_{IOP} if for any $\mathbf{x} \notin \mathcal{L}$ it holds that:

$$\Pr \left(\begin{array}{l} \mathcal{V}^{(\pi^{(0)}, \dots, \pi^{(k)})}(r_1, \dots, r_k) = 1, \\ \pi^{(0)} \leftarrow \mathcal{P}(\mathbf{x}, \mathbf{w}) \\ \dots \\ \pi^{(k)} \leftarrow \mathcal{P}(\mathbf{x}, \mathbf{w}, r_1, \dots, r_k) \end{array} \right) \leq \epsilon_{IOP}$$

An IOP is knowledge sound with knowledge soundness error κ_{IOP} with extraction time $t_{\mathcal{E}}$, if there exists a PPT extractor $\mathcal{E}$ such that for any $\mathcal{P}$ and $\mathbf{x}$:

$$\Pr \left(\begin{array}{l} \mathcal{V}^{(\pi^{(0)}, \dots, \pi^{(k)})}(r_1, \dots, r_k) = 1 \wedge (\mathbf{x}, \mathbf{w}) \notin \mathcal{R}, \\ \pi^{(0)} \leftarrow \mathcal{P}(\mathbf{x}, \mathbf{w}) \\ \dots \\ \pi^{(k)} \leftarrow \mathcal{P}(\mathbf{x}, \mathbf{w}, r_1, \dots, r_k) \\ \mathbf{w} \leftarrow \mathcal{E}^{\mathcal{P}}(\mathbf{x}, \pi^{(0)}, \dots, \pi^{(k)}, r_1, \dots, r_k) \end{array} \right) \leq \kappa_{IOP}$$

Performance Characteristics. In this work, we consider the following performance characteristics of an IOP: (1) The number of rounds $k(|\mathbf{x}|)$ (2) The size $l(|\mathbf{x}|)$ of the concatenation of all strings $(\pi^{(1)}, \dots, \pi^{(k)})$, (3) The total number of queries $q(|\mathbf{x}|)$ of $\mathcal{V}$ and finally (4) $pt(|\mathbf{x}|)$ and $vt(|\mathbf{x}|)$ the prover and verification time.

1.3. Round-by-Round (Knowledge) Soundness

Definition 3 (Round-by-Round Soundness). An IOP for the relation $\mathcal{R}$ has round-by-round soundness error ϵ_{rbr} if there exists a doomed set $\mathcal{D}$ of partial and complete transcripts such that:

- 1) If $\mathbf{x} \notin \mathcal{L}_{\mathcal{R}}$, then $(\mathbf{x}, \emptyset) \in \mathcal{D}$.
- 2) For a complete proof transcript $\mathbf{t}$, if $(\mathbf{x}, \mathbf{t}) \in \mathcal{D}$ then the verifier rejects.
- 3) For every $i \in [k]$, and any partial transcript $\mathbf{t}_i$, such that $(\mathbf{x}, \mathbf{t}_i) \in \mathcal{D}$, then for any possible reply $\pi^{(i+1)}$ it holds that:

$$\Pr_{r_{i+1} \xleftarrow{\$} \mathbb{F}} [(\mathbf{t}_i, \pi^{(i+1)}, r_{i+1}) \notin \mathcal{D}] \leq \epsilon_{rbr}$$

Definition 4 (Round-by-Round Knowledge Soundness). An IOP for the relation $\mathcal{R}$ has round-by-round knowledge soundness error κ_{rbr} if there exists a PPT extractor $\mathcal{E}$ and a doomed set $\mathcal{D}$ of partial and complete transcripts such that:

- 1) $(\mathbf{x}, \emptyset) \in \mathcal{D}$ for all possible input.
- 2) For any input $\mathbf{x}$ and complete proof transcript $\mathbf{t}$, if $(\mathbf{x}, \mathbf{t}) \in \mathcal{D}$ then the verifier rejects.
- 3) For every $i \in [k]$, and any partial transcript $\mathbf{t}_i$, such that $(\mathbf{x}, \mathbf{t}_i) \in \mathcal{D}$, then for any possible reply $\pi^{(i+1)}$ it holds that if:

$$\Pr_{r_{i+1} \xleftarrow{\$} \mathbb{F}} [(\mathbf{t}_i, \pi^{(i+1)}, r_{i+1}) \notin \mathcal{D}] > \kappa_{rbr}$$

Then, the extractor outputs a witness $\mathbf{w}$ such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$.

In this work, we will rely on the BCS transformation that compiles any IOP to a non-interactive protocol so that the following hold:

Theorem of [3], [4], [5] For any IOP with $l(\mathbf{x}), q(\mathbf{x}), \epsilon_{rbr}, \kappa_{rbr}$, there exists a polynomial-time transformation BCS such that for every relation $\mathcal{R}$, random oracle $H : \{0, 1\}^* \rightarrow \{0, 1\}^k$ and query bound $Q \in \mathbb{F}$ it produces a non-interactive proof system for $\mathcal{R}$ with:

- Adaptive soundness error $\epsilon_{fs} \leq Q\epsilon_{rbr} + 3(Q^2 + 1)/2^k$ and adaptive knowledge soundness error of $\kappa_{fs} \leq Q\kappa_{rbr} + 3(Q^2 + 1)/2^k$.
- Adaptive soundness error $\epsilon_{qfs} = \Theta(Q\epsilon_{fs})$ and adaptive knowledge soundness error of $\kappa_{qfs} = \Theta(Q\kappa_{fs})$ for $Q\text{-}\mathcal{O}(q(\mathbf{x}) \log l(\mathbf{x}))$ -query adversaries.

1.4. Definition of a coSNARK

Definition 5. The set of PPT algorithms $\{\mathcal{G}, \text{Prove}, \text{Verify}\}$ is a coSNARK for an NP relationship $\mathcal{R}$ and secure up to $t \leq N - 1$ corruptions, if the following properties hold:

- 1) *Completeness:* For any security parameter λ , relationship $\mathcal{R}$ and $(\mathbf{x}, \mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_N)) \in \mathcal{R}$, it holds that:

$$\Pr \left(\begin{array}{c} pk, vk \leftarrow \mathcal{G}(1^\lambda, \mathcal{R}) \\ \pi \leftarrow \text{Prove}_{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(pk, \mathbf{x}, (\mathbf{w}_1, \dots, \mathbf{w}_N)) \\ 1 = \text{Verify}(vk, \mathbf{x}, \pi) \end{array} \right) = 1$$

Where $\langle \cdot, \cdot \rangle$ denotes an interactive protocol between parties, t is the proof transcript between all provers and the verifier and 1 indicates that $\mathcal{V}$ accepted the proof, namely, t .

- 2) *Knowledge Soundness:* For any λ , NP relationship $\mathcal{R}$, instance $\mathbf{x}$ and $pk, vk \leftarrow \mathcal{G}(1^\lambda, \mathcal{R})$ there exists a PPT extractor $\mathcal{E}$ that has (rewinding) access to all provers $\vec{\mathcal{P}}$ such that the following probability is negligible:

$$\Pr \left(\begin{array}{c} 1 = \text{Verify}(vk, \mathbf{x}, \pi) \wedge (\mathbf{x}, \mathbf{w}) \notin \mathcal{R} : \\ \pi \leftarrow \text{Prove}_{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(pk, \mathbf{x}, (\mathbf{w}_1, \dots, \mathbf{w}_N)), \\ \mathbf{w} \leftarrow \mathcal{E}_{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(pk) \end{array} \right)$$

- 3) *t-zero knowledge:* A collaborative argument of knowledge is t-zero knowledge if for any PPT adversary $\mathcal{A}$ that corrupts at most t provers as well as the verifier, then, there exists a PPT simulator $\mathcal{S}$ such that the following views are computationally indistinguishable:

- The view $\text{View}_{\mathcal{A}}^R(\mathbf{x}, \mathbf{w}, pk, vk)$ of the adversary, that is the list of messages all corrupted provers and the verifier receive when interacting with the honest provers.
- The view $\text{View}_{\mathcal{A}}^S(\mathbf{x}, pk, vk)$ of the adversary when interacts with simulator $\mathcal{S}$.

- 4) *Succinctness:* The proof size and verification times are poly-logarithmic to $|\mathbf{w}|$.

2. Proof of Theorem 1

Theorem 1. *Construction 2* is a (t, l) -zero knowledge collaborative code for $\text{RS}^{\otimes 2}$, with $t \leq k' - k + 1$ and $l = m' - m$.

Furthermore, assuming that $k' = \Theta(N)$, the above protocol has per-party time complexity of $\mathcal{O}(\frac{k'm'}{N} \log k'm')$, one (online) round with a total communication complexity of $\approx Nm' \log |\mathbb{F}|$ bits. In addition, our protocol is complete, that is, it produces the same result as if we run the encoding algorithm locally using the same randomness.

We first provide a performance analysis of **Construction 2**. Next we show that Π^{Enc} is complete and finally we show that our collaborative code satisfies (t, l) -zero knowledge, where $l = (m' - m)$.

Performance Analysis. Throughout our analysis we assume that $k' = \Theta(N)$, which should be the case for the application we focus. With that in mind, observe that from the analysis of the protocol of [6], the instantiation of $\mathcal{F}_{rand}$ requires one round of interaction, it has total communication complexity of $\mathcal{O}(Nl) = \mathcal{O}(k'l)$ and per-party time complexity of $\mathcal{O}(\frac{k'l}{N})$.

For the rest of this part we will focus on the online complexity of the protocol. For this, observe that at step 1, each party has to perform $\frac{m}{N}$ FFTs of size at most N requiring $\mathcal{O}(\frac{m}{N} \cdot (N \log N)) = \mathcal{O}(m' \log k')$ field operations. At step 3, each party has to perform one iFFT and an FFT of size at most $\mathcal{O}(M) = \mathcal{O}(m')$, which requires $\mathcal{O}(m' \log m')$ field operations. Hence, the total complexity of each party is $\mathcal{O}(m' \log(k'm')) = \mathcal{O}(\frac{m'k'}{N} \log(k'm'))$. Finally, observe that the protocol needs only one round of communication (step 2) in which the total communication complexity is $\mathcal{O}(N \cdot (N \frac{m'}{N})) = \mathcal{O}(Nm') = \mathcal{O}(k'm')$.

Remark 1. In many applications, especially in collaborative zkSNARKs such as private delegation of proof generation, each party $\mathcal{P}_i$ will already be given the i -th row $\mathbf{X}'$ (e.g., provided by the delegator who must secret share its extended witness). In these cases, steps 1 and 2 of **Construction 2** are not needed, which leads to no communication overheads and a per-party time complexity of $\mathcal{O}(\frac{k'm'}{N} \log m')$.

Completeness. Next, we show that for any $\mathbf{X}$ each party $\mathcal{P}_i$ will compute the i -th row of the $\text{RS}^{\otimes 2}$ codeword of $\mathbf{X}$. In particular, we will show that the row $\mathbf{C}[i, :]$ computed at step 3 of **Construction 2** is equal to $[p(\omega_1^i, \omega_2^1), \dots, p(\omega_1^i, \omega_2^M)]$, where $p : \mathbb{F}^2 \rightarrow \mathbb{F}$ the bivariate polynomial such that $p(z_1^i, z_2^j) = \tilde{\mathbf{X}}[i, j]$. First, due to the completeness of the $\mathcal{F}_{rand}$ functionality, the vector $\mathbf{R}'[i, :]$ that $\mathcal{P}_i$ holds at the end of the pre-processing phase must be equal to $[p_r^{(1)}(\omega_1^i), \dots, p_r^{(m'-m)}(\omega_1^i)]$. Next, by the definition of $\mathbf{X}$, it must hold that $c^{(i,j)}(X) = c^{((i-1)\frac{m}{N} + j)}(X)$, where $c^{(i,j)}(X)$ the polynomial that interpolates the j -th column of $\tilde{\mathbf{X}}_i$. Furthermore, at the end of step 2, each party $\mathcal{P}_i$ will hold the i -th row of $\mathbf{X}'_j$ of any other party $\mathcal{P}_j$. The concatenation of those rows equals to $[c^{(1,1)}(\omega_1^i), \dots, c^{(N,m)}(\omega_1^i)] = [c^{(1)}(\omega_1^i), \dots, c^{(m)}(\omega_1^i)]$. By adding $\mathbf{R}[i, :]$ we get $\mathbf{X}'[i, :]$ as defined in the main body. Since $\mathbf{X}'[i, :]$ is correctly computed, it follows that $\mathbf{C}[i, :]$ will also be correct due to the correctness of the iFFT and FFT algorithms.

(t, l) -zero knowledge. We first construct a simulator $\mathcal{S}$ that simulates the view of an $\mathcal{A}$ that controls up to t corrupted parties $\mathcal{P}^* = \{\mathcal{P}_{j_1}, \dots, \mathcal{P}_{j_t}\}$ and the verifier $\mathcal{V}$. The simulator is given access to the random tape of $\mathcal{A}$ (and consequently the index set $\mathcal{I}$), the index set of corrupted parties $\mathcal{J} = \{j_1, \dots, j_t\} \in [N]^t$, as well as their inputs $(\mathbf{X}_{j_1}, \dots, \mathbf{X}_{j_t})$ and works as follows:

- 1) Let $\mathcal{I}_c = \{(\omega_1^j, z_2^j)\}_{j \in \mathcal{J}, i \in [m'] \setminus \mathcal{J}_{col}} \cup \{\omega_1^j, \omega_2^j\}_{j \in \mathcal{J}, i \in [M]}$, be the set of points the evaluations of which are obtained by the adversary at the end of the Π^{Enc} protocol (excluding what $\mathcal{A}$ already knows at the beginning of the protocol). Note that $\mathcal{J}_{col}$ are the indexes of the columns of $\mathbf{X}$, the corrupted parties hold at the beginning of the protocol. Furthermore, let $\mathcal{Y}_{\mathcal{I}_c \cup \mathcal{I}}$ be the set of all “valid” values that $\mathbf{X}'$ and $\mathbf{C}$ can have at these points. In practice, $\mathcal{Y}_{\mathcal{I}_c \cup \mathcal{I}}$ is the set of evaluations of all k', m' -degree bivariate polynomials p at $\mathcal{I}_c \cup \mathcal{I}$ conditioned that they evaluate at $(\tilde{\mathbf{X}}_{j_1}, \dots, \tilde{\mathbf{X}}_{j_t})$ on the points $\{z_1^i, z_2^j\}_{i \in [k'], j \in \mathcal{J}_{col}}$. The simulator samples an element of $\mathbf{Y} \xleftarrow{\$} \mathcal{Y}_{\mathcal{I}_c \cup \mathcal{I}}$ uniformly at random in the following way:

- a) $\mathcal{S}$ starts by computing the answers $\mathbf{c}$ to the queries $\mathcal{I}$. This is done as follows:
 - i) Parse $\mathcal{I}$ as a multi-set consisting of sets of queries that target the same column of $\mathbf{C}$, that is, $\mathcal{I}^{(j)} = \{(\omega_1^{i_1}, \omega_2^{j_1}), \dots, (\omega_1^{i_z}, \omega_2^{j_z})\}$ for all $\{i_1, \dots, i_z\} \in \mathcal{I}_1^{(j)}$ and all possible columns $j \in \mathcal{I}_2$. Note that the number of different queried columns as at most l , namely, $|\mathcal{I}_2| \leq l$.
 - ii) Next, iterate through all $\mathcal{I}^{(i)}$ and sample a random “valid” column $\mathbf{C}[:, i]$ that it interpolates to a polynomial of degree k' . Store the points that correspond to the queries $\mathcal{I}^{(i)}$ and $\mathbf{C}[j, i]$, for all $j \in \mathcal{J}$ and discard all other points.
- b) Sample uniformly at random the rows $\mathbf{X}'_i[j, :]$ sent by the honest parties to $\mathcal{J}$. Furthermore, if $|\mathcal{I}_2| < l$, sample $\mathbf{R}'[j, i]$, for all $j \in \mathcal{J}, i \in [l - |\mathcal{I}_2|]$ uniformly at random. Next, use: (1) $\mathbf{X}'_i[j, :]$, for all $i \in [N], j \in \mathcal{J}$, (2) the values $\{\mathbf{C}[j, i]\}_{j \in \mathcal{J}, i \in \mathcal{I}^{(2)}}$ computed at step a.ii and (3) the sampled points of $\mathbf{R}'$ (if any) to interpolate the m' -degree polynomials $r^{(j)}(Y)$, for all $j \in \mathcal{J}$. Finally use these polynomials to compute the rest of the points of $\mathbf{R}'[j, :]$.

- 2) Provided $\mathbf{c}, \mathbf{X}'_i[j, :]$, for all $i \in [N] \setminus \mathcal{J}$ and $j \in \mathcal{J}$ and $\mathbf{R}'[j, :]$, $j \in \mathcal{J}$, $\mathcal{S}$ simulates **Construction 2** as follows:

- a) $\mathcal{S}$ simulates $\mathcal{F}_{rand}$ functionality by sending $\mathbf{R}[j_i, :]$ for all $j_i \in \mathcal{J}$ to the adversary. Finally, $\mathcal{S}$ sets $V_1 = \{\mathbf{R}'[j, :]\}_{j \in \mathcal{J}}$.
- b) $\mathcal{S}$ simulates step 2 by sending $\mathbf{X}'_i[j, :]$, for all $i \in [N] \setminus \mathcal{J}$ to the adversary and setting $V_2 = \{\mathbf{X}'_i[j, :]\}_{i \in [N] \setminus \mathcal{J}, j \in \mathcal{J}}$.
- c) Finally, from $\mathbf{Y}$ the simulator retrieves and sends to the adversary the reply queries $\mathbf{c}$. Finally it sets $V_3 = \{\mathbf{Y}\}$.

What remains is to show that $View_{\mathcal{J}}^{\Pi^{Enc}}(\mathbf{x})$, parsed as

$\{V_1^*, V_2^*, V_3^*\}$ is indistinguishable to $S(\mathbf{x}_{j_1}, \dots, \mathbf{x}_{j_t}, \mathcal{I}) = \{V_1, V_2, V_3\}$, namely, show that $V_i \equiv V_i^*$, for $i = \{1, 2, 3\}$. The indistinguishability between the first two views comes directly from [7] as long as $t \leq k' - k + 1$. Consequently, we will focus on showing that $V_3 \equiv V_3^*$. As mentioned earlier, our analysis will focus on the case where $t = k' - k + 1$, but we can easily generalize our proof for $t < k' - k + 1$.

We start with the observation that the simulator samples $\mathbf{Y} = V_3$ uniformly at random from the set of valid answers $\mathcal{Y}_{\mathcal{I} \cup \mathcal{I}_c}$. Consequently, we need to show that V_3^* also follows a uniform distribution over $\mathcal{Y}_{\mathcal{I} \cup \mathcal{I}_c}$. In other words, we need to show that for any global input $\mathbf{X}$, and a k', m' -degree polynomial p that passes through $\{\mathbf{X}_j\}_{j \in [N] \setminus \mathcal{J}}$ and $\{\tilde{\mathbf{X}}_j\}_{j \in \mathcal{J}}$ it holds that:

$$Pr[p|_{\mathcal{I} \cup \mathcal{I}_c} = V_3] = \frac{1}{|\mathcal{Y}_{\mathcal{I} \cup \mathcal{I}_c}|}$$

To achieve this, we first prove the following lemma:

Lemma 2. *Let $\mathcal{I}'$ be a multiset, such that $\mathcal{I} \subset \mathcal{I}'$ which is defined as follows: If a set $\mathcal{I}^{(j)} \in \mathcal{I}$ has more than k elements, then $\mathcal{I}'^{(j)} = \mathcal{I}^{(j)}$ otherwise fill $\mathcal{I}'^{(j)}$ with indexes at the same column so that $|\mathcal{I}'^{(j)}| = k$. Finally, if $|\mathcal{I}_2| < l$ (where $\mathcal{I}_2$ the unique column indexes of the query set $\mathcal{I}$), fill $\mathcal{I}'$ with $l - |\mathcal{I}_2|$, k -sized sets of indexes, such that all sets $\mathcal{I}'^{(j)}$ contain indexes over a distinct column. For any $\mathbf{Y} \in \mathcal{Y}_{\mathcal{I}_c \cup \mathcal{I}'}$, there exists a unique k', m' -degree bivariate polynomial $p : \mathbb{F}^2 \rightarrow \mathbb{F}$ that passes through $\{\mathbf{X}_j\}_{j \in [N] \setminus \mathcal{J}}$ and $\{\tilde{\mathbf{X}}_j\}_{j \in \mathcal{J}}$ and $\mathbf{Y}$.*

Proof. Observe that given $\{\mathbf{X}_j\}_{j \in [N] \setminus \mathcal{J}}$ and $\{\tilde{\mathbf{X}}_j\}_{j \in \mathcal{J}}$ and $\mathbf{Y}$ we can interpolate the polynomial $p : \mathbb{F}^2 \rightarrow \mathbb{F}$ as follows: First, organize $\mathcal{I}' \cup \mathcal{I}_c$ into sets of indexes that belong to the same column. Then, filter from $\mathcal{I}' \cup \mathcal{I}_c$ all such sets with size smaller than k' . By definition of $\mathcal{I}_c$ and $\mathcal{I}'$ we remain with $m - t \frac{m}{N} + l = m' - t \frac{m}{N}$ such sets. Then, we use them to interpolate the k' -degree polynomials $c^{(j)}(X)$, where $j \in \mathcal{I}'^{(2)}$. Next, using $\{\tilde{\mathbf{X}}_j\}_{j \in \mathcal{J}}$ interpolate the remaining $t \frac{m}{N}$ polynomials $c^{(j)}(X)$, $j \in \mathcal{J}_{col}$. Finally, since the number of “partial” polynomials is m' , we can interpolate a m' -degree polynomial $q_i(Y)$ from the coefficients of each monomial of these polynomials thus constructing $p(X, Y) = \sum_{i \in [k']} X^i q_i(Y)$. $\square$

Provided Lemma 2, it is not hard to see that the probability, we obtain $\mathbf{Y} \in \mathcal{Y}_{\mathcal{I}_c \cup \mathcal{I}'}$ from a polynomial p that passes through $\{\mathbf{X}_j\}_{j \in [N] \setminus \mathcal{J}}$ and $\{\tilde{\mathbf{X}}_j\}_{j \in \mathcal{J}}$ is exactly:

$$Pr[p|_{\mathcal{I}_c \cup \mathcal{I}'} = \mathbf{Y}] = \frac{1}{|\mathcal{Y}_{\mathcal{I}_c \cup \mathcal{I}'}|} = \frac{1}{|\mathbb{F}|^{m'k' - mk - \frac{tm}{N}(k' - k)}}$$

Furthermore, using the definition $\mathcal{I}'$ from Lemma 2, we can make the following observations: (1) $\mathcal{I} \cup \mathcal{I}_c \subset \mathcal{I}' \cup \mathcal{I}_c$ and (2) $\mathcal{Y}_{\mathcal{I}' \cup \mathcal{I}_c} = \mathcal{Y}_{\mathcal{I} \cup \mathcal{I}_c} \times \mathcal{Y}_{\mathcal{I}' \setminus \mathcal{I}}$ and that is because $\mathcal{I}' \cap \mathcal{I}_c = \emptyset$ and $\mathcal{Y}_{\mathcal{I}'} = \mathcal{Y}_{\mathcal{I}} \times \mathcal{Y}_{\mathcal{I}' \setminus \mathcal{I}}$. Based on these observations, we have that:

$$\begin{aligned}
& \Pr[p|_{\mathcal{I}' \cup \mathcal{I}_c} = \mathbf{Y}] = \\
& \Pr[(p|_{\mathcal{I} \cup \mathcal{I}_c} = \mathbf{Y}^{(1)}) \cup (p|_{\mathcal{I}' \setminus \mathcal{I}} = \mathbf{Y}^{(2)})] = \\
& \Pr[(p|_{\mathcal{I} \cup \mathcal{I}_c} = \mathbf{Y}^{(1)})] \cdot \frac{1}{|\mathcal{Y}_{(\mathcal{I}' \setminus \mathcal{I})}|} = \\
& \Pr[(p|_{\mathcal{I} \cup \mathcal{I}_c} = \mathbf{Y}^{(1)})] \cdot \frac{1}{|\mathbb{F}|^{|\mathcal{I}' \setminus \mathcal{I}|}}
\end{aligned}$$

Consequently:

$$\Pr[p|_{\mathcal{I} \cup \mathcal{I}_c} = \mathbf{Y}^{(1)}] = \Pr[p|_{\mathcal{I}' \cup \mathcal{I}_c} = \mathbf{Y}] \cdot |\mathbb{F}|^{|\mathcal{I}' \setminus \mathcal{I}|} = \frac{1}{|\mathbb{F}|^{m'k' - mk - \frac{tm}{N}(k' - k) - |\mathcal{I}' \setminus \mathcal{I}|}}$$

3. On the Foldability of $\text{RS}^{\otimes 2}$ Codes

In this section, we show that $\text{RS}^{\otimes 2}$ are *foldable*. In more detail, and similarly to standard RS codes, we can define a folding function $Fold$ that takes as input an element $a \in \mathbb{F}$ and a function $f : \mathcal{L}'_1 \times \mathcal{L}'_2^{(i)} \rightarrow \mathbb{F}$ and “hashes” it into a function of half its size. More precisely, $Fold(a, f) : \mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)} \rightarrow \mathbb{F}$, where $|\mathcal{L}'_2^{(i+1)}| = |\mathcal{L}'_2^{(i)}|/2$. For any $a \in \mathbb{F}$, and $y \in \mathcal{L}'_2^{(i+1)}$ and $y', -y' \in \mathcal{L}'_2^{(i)}$ such that $y = y'^2 = (-y')^{2^1}$, we define $Fold(a, f)$ as follows:

$$\begin{aligned}
Fold(a, f)(x, y) &= \frac{f(x, y') + f(x, -y')}{2} + \\
& a \cdot \frac{f(x, y') - f(x, -y')}{2y'}
\end{aligned}$$

Based on the above function, we make the following Lemma that will be necessary for the completeness and soundness of our collaborative IOPP:

Lemma 3. *For the Fold function of the above equation the following holds:*

- 1) If $f \in \text{RS}^{\otimes 2}[\mathbb{F}, \mathcal{L}'_1 \times \mathcal{L}'_2^{(i)}, (k, k', N), (m, m', M)]$, then, for any $a \in \mathbb{F}$, $Fold(a, f) \in \text{RS}^{\otimes 2}[\mathbb{F}, \mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}, (k, k', N), (\frac{m}{2}, \frac{m'}{2}, \frac{M}{2})]$
- 2) Let $\delta < \frac{\Delta}{3}$, where Δ the relative distance of the $\text{RS}^{\otimes 2}$ code. If $\Delta(f, \mathbf{C}_i) > \delta$ where $\mathbf{C}_i \in \text{RS}^{\otimes 2}[\mathbb{F}, \mathcal{L}'_1 \times \mathcal{L}'_2^{(i)}, (k, k', N), (m, m', M)]$, then for a random $a \in \mathbb{F}$ it holds that:

$$\Pr[\Delta(Fold(a, f), \mathbf{C}_{i+1}) \leq \delta] \leq \frac{n}{|\mathbb{F}|}$$

Where $n = |\mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}|$ $\mathbf{C}_{i+1} \in \text{RS}^{\otimes 2}[\mathbb{F}, \mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}, (k, k', N), (\frac{m}{2}, \frac{m'}{2}, \frac{M}{2})]$

Proof. To prove the first claim, let $p : \mathbb{F}^2 \rightarrow \mathbb{F}$ be the polynomial of individual degrees k' and m' respectively, that interpolates f . Furthermore, let $p_o, p_e : \mathbb{F}^2 \rightarrow \mathbb{F}$ be the polynomials from p that contain even and odd powers respectively on the Y variable. Similarly with the univariate case, we can define the two polynomials as $p_o(X, Y) = \frac{p(X, Y) + p(X, -Y)}{2}$

and $p_e(X, Y) = \frac{p(X, Y) - p(X, -Y)}{2Y}$. Next, we define the polynomial p^* of individual degrees k' and $\frac{m'}{2}$ respectively such that $p^*(X, Y') = p'_o(X, Y) + ap'_e(X, Y) = p_o(X, Y^2) + ap_e(X, Y^2)$. It is not hard to see that $p^*|_{\mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}} = Fold(f, a)$, and hence $Fold(f, a) \in \text{RS}^{\otimes 2}[\mathbb{F}, \mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}, (k, k', N), (\frac{m}{2}, \frac{m'}{2}, \frac{M}{2})]$. That is, because for each $(x, y) \in \mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}$, $p^*(x, y) = p'_o(x, y) + ap'_e(x, y) = p_o(x, y') + ap_e(x, y') = \frac{f(x, y') - f(x, -y')}{2} + a \frac{f(x, y') - f(x, -y')}{2y'} = Fold(a, f)(x, y)$

To prove the second property, we use Lemma 1. First, let $\mathbf{u} = Fold(0, f)$, $\mathbf{u}^* = Fold(1, f) - Fold(0, f)$. Note that $Fold(a, f) = \mathbf{u} + a\mathbf{u}^*$. In addition, let $|\mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}| = n$. Assume towards contradiction that $\Pr[\Delta(Fold(a, f), \mathbf{C}_{i+1}) \leq \delta] = \Pr[\Delta(\mathbf{u} + a\mathbf{u}^*, \mathbf{C}_{i+1}) \leq \delta] > \frac{n}{|\mathbb{F}|}$.

By Lemma 1, we have that $\Delta(\{\mathbf{u}, \mathbf{u}^*\}, \mathbf{C}_{i+1}) \leq n\delta$. This implies that there should exist $\mathbf{w}, \mathbf{w}^* \in \text{RS}^{\otimes 2}[\mathbb{F}, \mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}, (k, k', N), (\frac{m}{2}, \frac{m'}{2}, \frac{M}{2})]$, and a set of indexes $\mathcal{T}$, such that $|\mathcal{T}| \geq n(1 - \delta)$ and $\mathbf{w}|_{\mathcal{T}} = \mathbf{u}|_{\mathcal{T}}$ and $\mathbf{w}^*|_{\mathcal{T}} = \mathbf{u}^*|_{\mathcal{T}}$. Let $p_w, p_{w^*} : \mathbb{F}^2 \rightarrow \mathbb{F}$ the bivariate polynomials obtained by interpolating $\mathbf{w}, \mathbf{w}^*$. Note that these polynomials have individual degree of at most k' and $m'/2$ respectively. Furthermore, we define the polynomials $\hat{p}(X, W, Y) = p_w(X, Y) + Wp_{w^*}(X, Y)$ and $p^*(X, Y) = \hat{p}(X, Y, Y^2)$. Note that the latter has individual degree of at most k' and m' .

Given those polynomials, we will show that f agrees with $p^*(X, Y)$ at all points $(x, y'), (x, -y')$, where $y = y'^2 = (-y')^2$, and $(x, y) \in \mathcal{T}$. For this, first observe that $\hat{p}(x, 0, y) = Fold(0, f)(x, y)$ and $\hat{p}(x, 1, y) = Fold(1, f)(x, y)$. Since both $\hat{p}(x, W, y)$ and $Fold(W, f)(x, y)$ are lines and agree at two points, it will hold that $\hat{p}(x, W, y) = Fold(W, f)(x, y)$. Consequently $p^*(x, y') = \hat{p}(x, y', y) = Fold(y', f) = f(x, y')$. Likewise, we argue about $(x, -y')$. Let $p^*|_{\mathcal{L}'_1 \times \mathcal{L}'_2^{(i+1)}} = \mathbf{C}_i$, then $\Delta(f, \mathbf{C}_i) \leq 1 - \frac{2|\mathcal{T}|}{2n} \leq \delta$ which contradicts our assumption. Consequently, $\Delta(f, \mathbf{C}_i) > \delta$. $\square$

4. Proofs of Section 5

4.1. Definition of Collaborative PCS

In this section, we give the definition of a collaborative PCS. We specifically focus on coPC schemes for multilinear polynomials but we can trivially extend the definition for multi-variate polynomials.

Definition 6 (Collaborative Polynomial Commitment Scheme). *A collaborative PCS is a tuple of algorithms (Gen, Commit, Eval, Verify) defined as follows:*

- $\mathbf{pk}, \mathbf{vk} \leftarrow \text{Gen}(1^\lambda, M)$: Given the security parameter λ and the maximum size (i.e., number of coefficients) of the polynomial to be committed, generate a proving and verification key.
- $C_f \leftarrow \text{Commit}^{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(\mathbf{pk}, \mathbf{x}_1, \dots, \mathbf{x}_N)$: An MPC protocol between N provers $\mathcal{P} = \{\mathcal{P}_1, \dots, \mathcal{P}_N\}$. Each

1. In practice, $y' = y$ and $-y' = y + \frac{|\mathcal{L}'_2^{(i)}|}{2}$

$\mathcal{P}_i$ holds its private input $\mathbf{x}_i$ and all provers collaborate to output a polynomial commitment of $f \in \mathbb{F}^{\log M} \rightarrow \mathbb{F}$, the multi-linear extension of $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_N) \in \mathbb{F}^M$.

- $\pi, y \leftarrow \text{Eval}^{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(\mathbf{pk}, \mathbf{x}_1, \dots, \mathbf{x}_N, \mathbf{t})$: An MPC protocol between the provers for generating a proof π showing that $f(\mathbf{t}) = y$.
- $0, 1 \leftarrow \text{Verify}(\mathbf{vk}, C_f, \pi, \mathbf{t}, y)$: Given the PC commitment C_f , an evaluation proof and the evaluation pair $(\mathbf{t}, y)$, the algorithm returns 1 if and only if $f(\mathbf{t}) = y$

A collaborative PCS must satisfy the following:

- **Completeness:** For all $(\mathbf{x}_1, \dots, \mathbf{x}_N), \mathbf{t} \in \mathbb{F}^{\log M}$ and y such that $f(\mathbf{t}) = y$, it holds that:

$$\Pr \left(\begin{array}{l} \mathbf{pk}, \mathbf{vk} \leftarrow \text{Gen}(1^\lambda, M) \wedge \\ C_f \leftarrow \text{Commit}^{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(\mathbf{pk}, \mathbf{x}_1, \dots, \mathbf{x}_N) \wedge \\ \pi \leftarrow \text{Eval}^{\{\mathcal{P}_1, \dots, \mathcal{P}_N\}}(\mathbf{pk}, \mathbf{x}_1, \dots, \mathbf{x}_N, \mathbf{t}) \wedge \\ 1 \leftarrow \text{Verify}(\mathbf{vk}, C_f, \pi, \mathbf{t}, y) \end{array} \right) = 1$$

- **Knowledge Soundness:** For all $(\mathbf{x}_1, \dots, \mathbf{x}_2)$ and for any set of PPT adversaries $\mathcal{P}^* = \{\mathcal{P}_1^*, \dots, \mathcal{P}_N^*\}$, there exists a PPT extractor $\mathcal{E}$ with the ability to rewind $\mathcal{P}^*$, such that:

$$\Pr \left(\begin{array}{l} \mathbf{pk}, \mathbf{vk} \leftarrow \text{Gen}(1^\lambda, M) \wedge \\ C_f, \mathbf{t}, y, \pi \leftarrow \mathcal{P}^*(\mathbf{pk}, \mathbf{vk}) \wedge \\ \mathbf{x} \leftarrow \mathcal{E}^{\mathcal{P}^*}(\mathbf{pk}, \mathbf{vk}) : \\ 1 \leftarrow \text{Verify}(\mathbf{vk}, C_f, \pi, \mathbf{t}, y) \wedge \\ f(\mathbf{t}) \neq y \vee C_f \neq \text{Commit}(\mathbf{pk}, \mathbf{x}) \end{array} \right) < \text{negl}(\lambda)$$

- **Honest Verifier t-Zero-Knowledge:** For any PPT adversary controlling at most t adversaries $\{\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_t}\}$ as well as the verifier $\mathcal{V}$, there exists a PPT simulator $\mathcal{S}$ (with oracle access to the multi-linear extension of $\mathbf{x}$) such that:

$$\Pr(\text{Real}_{\mathcal{A}, f}(1^\lambda, \mathbf{pk}, \mathbf{vk})) \stackrel{c}{=} \Pr(\text{Ideal}_{\mathcal{A}, \mathcal{S}}(1^\lambda, \mathbf{pk}, \mathbf{vk})) \quad (1)$$

Where the **Ideal** and **Real** games are described below. We denote as $\mathbf{tr}$ the interaction transcript and $\text{View}_{\mathcal{P}^*, \mathcal{V}}^{\Pi_{\text{Eval}}}(\mathbf{x})$ is the accumulated view of all parties controlled by $\mathcal{A}$ when interacting via the protocol Π_{Eval} over the input $\mathbf{x}$.

Real $_{\mathcal{A}, f}(1^\lambda, \mathbf{pk}, \mathbf{vk})$:

- 1) $C_f \leftarrow \Pi_{\text{Commit}}^{\mathcal{P}}(\mathbf{pk}, \mathbf{x}_1, \dots, \mathbf{x}_N)$
- 2) $\mathbf{t} \leftarrow \mathcal{A}(\mathbf{pk}, \mathbf{vk}, C_f)$
- 3) $\pi \leftarrow \text{View}_{\mathcal{P}^*, \mathcal{V}}^{\Pi_{\text{Eval}}}(\mathbf{x})$
- 4) Output $b \leftarrow \mathcal{A}(1^\lambda, C_f, y, \pi, \mathbf{pk}, \mathbf{vk})$.

Ideal $_{\mathcal{A}, \mathcal{S}}(1^\lambda, \mathbf{pk}, \mathbf{vk})$:

- 1) $C_f \leftarrow \mathcal{S}_1(1^\lambda, M, \mathbf{x}_{i_1}, \dots, \mathbf{x}_{i_t})$
- 2) $\mathbf{t} \leftarrow \mathcal{A}(\mathbf{pk}, \mathbf{vk}, C_f)$
- 3) $\pi \leftarrow \mathcal{S}_2(\mathbf{pk}, \mathbf{vk}, \mathbf{x}_{i_1}, \dots, \mathbf{x}_{i_t}, \mathbf{r}, f(\mathbf{r}))$
- 4) Output $b \leftarrow \mathcal{A}(1^\lambda, C_f, y, \pi, \mathbf{pk}, \mathbf{vk})$.

4.2. Proof of Theorem 2

Theorem 2. Let $\delta = \min(\Delta/3, \max(\{\Delta(\mathbf{C}_r^{(i)}, \text{RS}_i^{\otimes 2})\}_i, \Delta(\mathbf{C}^{(1)}, \text{RS}_1^{\otimes 2})))$, where Δ the minimum relative distance of $\text{RS}^{\otimes 2}$. **Construction 3** is a coIOPP for showing that $\Delta(\mathbf{C}^{(1)}, \text{RS}_1^{\otimes 2}) \leq \delta$ with perfect completeness, soundness error of $\epsilon = (4(\log M - c)|\mathbf{C}^{(1)}|/|\mathbb{F}| + (1 - \delta)^l, \text{round-by-round (knowledge) soundness error of } \max(2|\mathbf{C}^{(1)}|/|\mathbb{F}|, (1 - \rho)^l))$, and honest-verifier and prover perfect t-zero knowledge supporting up to $t \leq k' - k + 1$ corruptions.

4.2.1. Performance Analysis. We start by analyzing the performance of the online phase of our coIOPP. Let $L = km$ be the size of $\mathbf{X}$. It is not hard to see that the total number of communication rounds between the verifier is $k_V(L) = \mathcal{O}(\log L)$ while the number of communication rounds only between the provers is $k_P(L) = 1$ which happens at step 1 of the commit protocol. Next, the total proof size $l(L) = |\mathbf{C}^{(\log M - c - 1)}| + \sum_{i \in [\log M - c - 1]} |\mathbf{C}^{(i)}| + |\mathbf{C}_r^{(i)}| \leq 4|\mathbf{C}^{(1)}| = \mathcal{O}(L)$, and $l_i(L) = \mathcal{O}(\frac{L}{N})$. The number of queries and the verification time are both $q(L, N) = vt(L, N) = \mathcal{O}(l \log L + N2^c)$. The $l \log L$ factor derives from the first step of the query algorithm while the $N2^c$ factor comes from the second one. Note that after applying the optimization, both complexities can be reduced to $q(L, N) = vt(L, N) = \mathcal{O}(l \log L + N)$.

Finally, the per-prover time complexity is $pt_i(L, N) = \mathcal{O}(\frac{L}{N} \log L)$ and the total communication complexity is $pc(L = km, N) = Nm \log |\mathbb{F}|$ bits. In practice, prover time is dominated by step 1 while in the rest of the protocol (and in particular the commitment phase) the per-prover time complexity is $\mathcal{O}(l_i(L)) = \mathcal{O}(\frac{L}{N})$. Finally, the total communication cost comes from the Π^{Enc} algorithm.

4.2.2. Completeness Proof. From the completeness property of Π^{Enc} , $\mathbf{C}^{(1)}$ is a valid codeword of the message $\mathbf{X}$. Likewise, all codewords $\mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log M - c - 1)}$ are valid by the definition of $\mathcal{F}_{\text{rand}}^{\text{Enc}}$. Based on that and from Lemma 3, we get that $\mathbf{C}^{(2)}, \dots, \mathbf{C}^{(\log M - c)}$ would be valid codewords, and in particular, $\mathbf{C}^{(i)} \in \text{RS}_i^{\otimes 2}$, for all $i \in [\log M - c]$. Consequently, step 2 of the query algorithm will pass with probability 1. Finally, by the definition of the fold function for $\text{RS}^{\otimes 2}$, it is not hard to see that all queries of step 1 will pass with probability 1. That is because

$$\begin{aligned} & \mathbf{C}^{(j+1)}[(x, y)] = \text{Fold}(r_j, \mathbf{C}^{(j)} + b_j \mathbf{C}_r^{(j)})[(x, y)] = \\ & \frac{\mathbf{C}^{(j)}[(x, y')] + \mathbf{C}^{(j)}[(x, -y')] + b_j(\mathbf{C}_r^{(j)}[(x, y')] + \mathbf{C}_r^{(j)}[(x, -y')])}{2} + \\ & r_j \frac{\mathbf{C}^{(j)}[(x, y')] - \mathbf{C}^{(j)}[(x, -y')] + b_j(\mathbf{C}_r^{(j)}[(x, y')] - \mathbf{C}_r^{(j)}[(x, -y')])}{2y'} = \\ & \text{INTERPOLATE}(a_j, \mathbf{C}^{(j)}[(x, y)] + b_j \mathbf{C}_r^{(j)}[(x, y)], \mathbf{C}^{(j)}[(x, -y)] + \\ & \quad b_j \mathbf{C}_r^{(j)}[(x, -y)]) \end{aligned}$$

Remark 2. For the next sections, we simplify our analysis by assuming that provers send $\mathbf{C}^{(\log m' - c)}$ directly to the

verifier instead of further reducing it to $\mathbf{C}^{\log m'}$. Note that our proofs can be naturally extended to cover our final version.

4.2.3. Proof of t -Zero Knowledge. We first construct a simulator $\mathcal{S}$ that simulates the view of an adversary $\mathcal{A}$ controlling up to t corrupted provers $\mathcal{P}^* = \{\mathcal{P}_{j_1}, \dots, \mathcal{P}_{j_t}\}$ and the verifier $\mathcal{V}$. The simulator is given the indexes of the corrupted provers $\mathcal{J}$ as well as access to the random tape of $\mathcal{A}$. In particular, it holds all random challenges $\mathbf{a} = (a_1, \dots, a_{\log m' - c - 1})$, $\mathbf{b} = (b_1, \dots, b_{\log m' - c - 1})$ of $\mathcal{V}$ during the *Commit* phase and also all query indexes $\mathcal{I} = \mathcal{I}^{(1)} \cup \mathcal{I}_r^{(1)} \cup \dots \cup \mathcal{I}^{(\log m' - c - 1)} \cup \mathcal{I}^{(\log m' - c)}$, of $\mathcal{V}$ during the *Query* phase (where $\mathcal{I}^{(i)}, \mathcal{I}_r^{(i)}$ the queries done over $\mathbf{C}^{(i)}$ and $\mathbf{C}_r^{(i)}$ resp.). The simulator works as follows:

- 1) *Initialization Step.* First, it generates the set of replies $\mathbf{q}^{(1)}, \dots, \mathbf{q}^{(\log m' - c - 1)}, \mathbf{q}_r^{(1)}, \dots, \mathbf{q}_r^{(\log m' - c - 1)} \in \mathbb{F}^l$, where $\mathbf{q}^{(i)}, \mathbf{q}_r^{(i)}$, the query replies from $\mathbf{C}^{(i)}$ and $\mathbf{C}_r^{(i)}$ respectively. This is recursively done as follows: first it samples uniformly at random the replies $\mathbf{q}^{(1)}, \mathbf{q}_r^{(1)}$ from the “valid” reply sets $\mathcal{V}_{\mathcal{I}^{(1)}}$ and $\mathcal{V}_{\mathcal{I}_r^{(1)}}$ respectively (as done by the (t, l) -zero knowledge simulator of the collaborative code). Then, it uses $\mathbf{q}_r^{(1)}, \mathbf{q}^{(1)}$ to compute the elements of $\mathbf{q}^{(2)}$ on the indexes queried by the verifier at step $b.i$ of the *Query* algorithm (when $j = 1$) and obtains the rest of the elements of $\mathbf{q}^{(2)}$ (e.g., the elements queried at step $b.i$ when $j = 2$) by sampling them uniformly at random such that $\mathbf{q}^{(2)} \in \mathcal{V}_{\mathcal{I}^{(2)}}$. Finally, it samples $\mathbf{q}_r^{(2)}$ uniformly at random from $\mathcal{V}_{\mathcal{I}_r^{(2)}}$. Likewise, use $\mathbf{q}^{(2)}, \mathbf{q}_r^{(2)}$ to construct $\mathbf{q}^{(3)}$, and recursively repeat the same process until reaching $\mathbf{q}^{(\log m' - c)}$ that is equal to $\mathbf{C}^{(\log m' - c)}$. The simulator computes $\mathbf{q}^{(\log m' - c)}$ as follows. First, it computes the elements of $\mathbf{q}^{(\log m' - c)}$ that correspond to the queries of step $b.i$ and samples the rest of $\mathbf{q}^{(\log m' - c)}$ uniformly at random such that $\mathbf{q}^{(\log m' - c)} = \mathbf{C}^{(\log m' - c)} \in \text{RS}_{\log m' - c}^{\otimes 2}$. Let $p' : \mathbb{F}^2 \rightarrow \mathbb{F}$ be the bivariate polynomial of individual degree k' and 2^c , that interpolates $\mathbf{C}^{(\log m' - c)}$.
- 2) *Simulate Π^{Enc} .* $\mathcal{S}$ invokes the simulator of the Π^{Enc} protocol for $\mathbf{C}^{(1)}$ but also all codewords $\mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log m' - c - 1)}$ with the minor difference that in the first step the simulator sets the query answers to be $\mathbf{q}^{(1)}$ and $\mathbf{q}_r^{(i)}$ respectively and for the first $\log m' - c - 2$ masked codewords it uses a zero matrix. In addition, for $\mathbf{C}^{(\log m' - c)}$, it initializes the view of the corrupted parties so that it agrees with $\mathbf{C}^{\log m' - c}$. Let V_1 be the view of the adversary (excluding the replies), that is, the view of each corrupted party after step 1 of the *Commit* phase.
- 3) *Simulate the rest of *Commit* phase.* The simulator follows all the steps performed by the corrupted provers. Furthermore, it setups the oracles by filling all parts of honest provers with zeros except from the positions in $\mathcal{I}^{(j)}, \mathcal{I}_r^{(j)}$ where it fills them with elements in $\mathbf{q}^{(j)}$ and $\mathbf{q}_r^{(j)}$ respectively. Let V_2 be the set containing all

oracles.

- 4) Finally, the simulator sets the view to be $\{V_1, V_2, V_3 = \{\{\mathbf{q}^{(i)}, \mathbf{q}_r^{(i)}\}_{i \in [\log m' - c - 1]}, \mathbf{q}^{(\log m' - c)}\}\}$, where V_3 is the view of the verifier at the *Query* phase.

Let $\text{View}_{\mathcal{J}}(\mathbf{X}) = \{V_1^*, V_2^*, V_3^* = \{\{\mathbf{q}^{*(i)}, \mathbf{q}_r^{*(i)}\}_{i \in [\log m' - c - 1]}, \mathbf{q}^{*(\log m' - c)}\}\}$ be the real view of the adversary. We will show that $\{V_1^*, V_2^*, V_3^*\} \equiv \{V_1, V_2, V_3\}$. First, we omit V_2 and V_2^* as they are inherently indistinguishable. So, we focus on proving that the distributions of (V_1, V_3) and (V_1^*, V_3^*) are identical. Observe that (V_1, V_3) has been sampled uniformly at random (from the set of accepting views). Consequently, we will need to show that (V_1^*, V_3^*) also follows a uniform distribution.

To this end, we need to show that for any joint input $\mathbf{X}$, the probability **Construction 3** generates the accepting view (V_1, V_3) is $\frac{1}{|\mathcal{V}_t|}$ where $\mathcal{V}_t$ the set of all accepting views (for the specific verifier’s challenges). To achieve this, we need to show that (1) for any accepting view (V_1, V_3) , we can obtain a set of bivariate polynomials $p, p_r^{(1)}, \dots, p_r^{(\log m' - c)} : \mathbb{F}^2 \rightarrow \mathbb{F}$, of the desirable individual degrees, that interpolate the codewords $\mathbf{C}^{(1)}, \mathbf{C}_r^{(1)}, \dots, \mathbf{C}^{(\log m' - c)}$ and that (2) any accepting view has the same number of such polynomials.

To prove (1), we follow a recursive strategy in which we first find $p^{(1)} := p$ and $p_r^{(1)}$ and then use them to compute $p^{(2)}$. With that in mind, observe that since the verifier performs only l queries over $\mathbf{C}^{(1)}$ and due to its (t, l) -zero knowledge property, by Lemma 2, we know that there will always be a polynomial p that interpolates $\mathbf{X}$ and evaluates to $\mathbf{q}_1^{(1)}$ at $\mathcal{I}^{(1)}$. What remains, is to interpolate $\mathbf{C}_r^{(1)}$. For this observe that $\mathbf{C}_r^{(1)}$ is directly queried by the verifier at step $b.i$ on the first round and indirectly at step $b.i$ on the second round of **Construction 3** when $\mathcal{V}$ queries $\mathbf{C}^{(2)}$. Let $\mathcal{I}'^{(2)}$ be the index set of those last queries, and $\mathbf{q}'^{(2)}$ the replies. We add to $\mathbf{q}_r^{(1)}$ at most l pairs $(\mathbf{C}_r^{(1)}[x_i, y_i], \mathbf{C}_r^{(1)}[x_i, -y_i])$ sampled uniformly at random such that:

$$b_1^{-1}(\mathbf{q}'^{(2)}[i] - \text{INTERPOLATE}(\mathbf{C}^{(1)}[(x_i, y_i)], \mathbf{C}^{(1)}[(x_i, -y_i)])) \\ = \text{INTERPOLATE}(\mathbf{C}_r^{(1)}[(x_i, y_i)], \mathbf{C}_r^{(1)}[(x_i, -y_i)])$$

This is done by sampling all points of the form $\mathbf{C}_r^{(1)}[x_i, y_i]$ from a valid reply set $\mathcal{V}_{\{(x_1, y_1), \dots, (x_l, y_l)\}}$ and then use them to compute $\mathbf{C}_r^{(1)}[x_i, -y_i]$. Because the left-hand side of the above equation and $\mathbf{C}_r^{(1)}[x_i, y_i]$ are both valid it follows that $\mathbf{C}_r^{(1)}[x_i, -y_i]$ will also be valid²

At this point, note that the updated vector $\mathbf{q}_r^{(1)}$ will have $3l$ points. Since $\mathbf{C}_r^{(1)}$ is $(t, 3l)$ -zero knowledge, it guaranteed by Lemma 2 that there exists a polynomial $p_r^{(1)}$ with the desired degree that interpolates $\mathbf{C}_r^{(1)}$. Lastly, using p and $p_r^{(1)}$ evaluate $\mathbf{C}^{(2)}$ and $p^{(2)}$. Following recursively the above process, we compute $p_r^{(i)}$ for all $i \in \{2, \dots, \log m' - c - 2\}$.

2. Note that by the definition of the query algorithm the indexes of the sets $\mathcal{I}_r^{(1)} = \mathcal{I}_r^{(1)}$, $\{(x_1, y_1), \dots, (x_l, y_l)\}$ and $\mathcal{I}_r'^{(1)} = \{(x_1, -y_1), \dots, (x_l, -y_l)\}$ do not lie on the same columns of $\mathbf{C}_r^{(1)}$. Consequently, we can sample from $\mathcal{V}_{\mathcal{I}_r^{(1)}}$ and $\mathcal{V}_{\mathcal{I}_r'^{(1)}}$ independently.

Note that in order to compute $p_r^{(\log m' - c - 1)}$ we directly use all elements of $\mathbf{q}^{(\log m' - c)}$.

It remains to show that given (V_1, V_3) and an input $\mathbf{X}$, the number of polynomials $\{p^{(1)}, p_r^{(1)}, \dots, p_r^{(\log m' - c - 1)}\}$ we can find is fixed. First, let $\mathcal{U}$ be the vector space that contains all accepting views (V_1, V_3) . Furthermore, let $\mathbf{A} \cdot \mathbf{c} = \mathbf{v}$ be the linear equation where $\mathbf{v} \in \mathcal{V}_t$, $\mathbf{c}$ consists of the coefficients of all polynomials $p^{(1)}, p_r^{(1)}, \dots, p_r^{(\log m' - c)}$ and $\mathbf{A}$ the constraint system that evaluates to $\mathbf{v}$. Let $r_1 > \dim(\mathcal{V}_t)$ be the rank of $\mathbf{A}$. From (1) we know that $\mathbf{A}$ is solvable for any $\mathbf{v} \in \mathcal{V}_t$ and the number of possible solutions is constant and equal to $|\mathbb{F}|^{|\mathbf{c}| - r_1}$.

4.2.4. Proof of Soundness. We will first focus on proving the standard soundness of our IOPP protocol and postpone the proof for round-by-round (knowledge) soundness for the next section. Initially, we consider the case that $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) = \max(\Delta(\pi^{(1)}, \mathbf{C}^{(1)}), \Delta(\pi_r^{(1)}, \mathbf{C}^{(1)}), \dots, \Delta(\pi_r^{(\log m' - c - 1)}, \mathbf{C}^{(\log m' - c - 1)}))$. In the end, we will also discuss how to extend our proof for the general case. In particular, in the commitment phase, and for each round i , we define two bad events E_i^* and E_i as follows:

- 1) E_i^* : $\delta_i^* = \min(\Delta(\pi^{(i)} + \beta_i \pi_r^{(i)}, \frac{\Delta}{3}), \Delta(\text{Fold}(a_i, \pi^{(i)} + \beta_i \pi_r^{(i)}), \mathbf{C}^{(i+1)})) < \delta_i^*$.
- 2) E_i : $\delta_i = \min((\Delta(\pi^{(i)}, \mathbf{C}^{(i)}), \frac{\Delta}{3}))$, but $\Delta(\pi^{(i)} + b_i \pi_r^{(i)}, \mathbf{C}^{(i)}) < \delta_i$

From Claim 3 we know that E_i^* happens only with probability of at most $\frac{|\pi^{(1)}|}{|\mathbb{F}|}$. The same holds for E_i too, from Lemma 1. By taking the union bound of these probabilities, we get $\Pr[E = \bigcup_{i \in [\log m' - c]} (E_i \cup E_i^*)] = \frac{2(\log m' - c)|\pi^{(1)}|}{|\mathbb{F}|}$.

Next, assuming that conditions E do not hold, we calculate the rejection probability of the verifier at the query phase. We start with the modification done over the oracles as presented in the previous proofs. Here we note that we also modify the oracles $\pi_r^{(1)}, \dots, \pi_r^{(\log m' - c - 1)}$ so that they produce accepting paths to $\pi^{(\log m' - c)}$. Finally, following a similar argument with [8] we end up with a rejection probability of $\sum_{i \in [\log m' - c - 1]} \beta_i$, where $\beta_i = \Delta(\pi^{(i+1)}, \text{Fold}(a_i, \pi^{(i)} + b_i \pi_r^{(i)}))$. We next, find a lower bound for $\sum_{i \in [\log m' - c - 1]} \beta_i$.

To achieve this, we first show that $\beta^{(i+1)} \geq \delta_i - \delta_{i+1}$. From the triangular inequality, we have that:

$$\begin{aligned} & \Delta(\text{Fold}(a_i, \pi^{(i)} + b_i \pi_r^{(i)}), \mathbf{C}^{(i+1)}) \leq \\ & \Delta(\pi^{(i+1)}, \text{Fold}(a_i, \pi^{(i)} + b_i \pi_r^{(i)})) + \Delta(\pi^{(i+1)}, \mathbf{C}^{(i+1)}) \equiv \\ & \delta_i^* \leq \beta_{i+1} + \Delta(\pi^{(i+1)}, \mathbf{C}^{(i+1)}) = \beta_{i+1} + \delta_{i+1} \end{aligned}$$

Note that $\delta_{i+1} = \Delta(\pi^{(i+1)}, \mathbf{C}^{(i+1)})$, since otherwise the claim would hold trivially (see [8], [9] for more details). It remains to “link” δ_i^* and δ_i with an inequality. For this we observe that $\delta_i \leq \delta_i^*$. To prove this, we consider two cases. In the first case, we assume that $\delta_i^* = \frac{\Delta}{3} \geq \min(\Delta(\pi^{(i)}, \mathbf{C}^{(i)}), \frac{\Delta}{3}) = \delta_i$. In the second case, we assume

that $\delta_i^* = \Delta(\pi^{(i)} + b_i \pi_r^{(i)}, \mathbf{C}^{(i)})$. Then, because E does not happen, it holds that $\delta_i^* = \Delta(\pi^{(i)} + b_i \pi_r^{(i)}, \mathbf{C}^{(i)}) \geq \delta_i$. Hence, we conclude that:

$$\beta_{i+1} \geq \delta_i^* - \delta_{i+1} \geq \delta_i - \delta_{i+1}$$

Last but not least, observe that we have $\sum_{i \in [\log m' - c - 1]} \epsilon_i \geq \delta_1 - \delta_{\log m' - c}$. Note that $\delta_{\log m' - c} = 0$ since otherwise the verifier would reject, so $\sum_{i \in [\log m' - c - 1]} \epsilon_i \geq \delta_1 = \delta$. Hence, for l queries, the acceptance probability would be $(1 - \sum_{i \in [\log m' - c - 1]} \epsilon_i)^l \leq (1 - \delta)^l$.

Finally, to prove the soundness of our coIOPP when there is a masked oracle $\pi_r^{(i)}$ with larger hamming distance, we follow the same procedure but change the way we modify the oracles in the last step of the proof. In particular we allow modification on $\pi^{(1)}$ but not on $\pi_r^{(i)}$.

4.3. Proof of Theorem 3

Theorem 3. Assuming $\delta < \frac{\Delta}{3}$ and $t \leq k' - k + 1$ corrupted provers, then our construction is a coIOPP for the relation $\mathcal{R}'_{MLE}$ with perfect completeness, $\text{negl}(\lambda)$ soundness and knowledge soundness error with an expected polynomial-time extractor, round-by-round (knowledge) soundness errors of $\max(\frac{|\mathbf{C}^{(1)}|}{|\mathbb{F}|}, (1 - \delta)^l)$ and perfect honest-verifier and prover t -zero knowledge. Finally, by applying our BCS we get a coPCS that is transparent and post-quantum with $O(\frac{L}{N} \log L)$ per-prover time complexity, $O(l \log^2 L)$ proof size and verification time and total communication cost of $O(L)$, where $L = mn$ the size of the polynomial.

Before delving into the proofs, for completeness, we provide a description of our coIOPP for $\mathcal{R}'_{MLE}$.

4.3.1. More Detailed Description of our coIOPP. We denote with $p^*, p : \mathbb{F}^2 \rightarrow \mathbb{F}$ the bivariate polynomials that interpolate $\mathbf{X}$ and the “masked” input $\tilde{\mathbf{X}}$. In addition, we let $g^* : \mathbb{F}^{\log k m} \rightarrow \mathbb{F}$ and $g : \mathbb{F}^{\log k' m'} \rightarrow \mathbb{F}$ be the multi-linear extensions of the coefficients of p^* and p .

Preprocessing Step. In this step, provers run a protocol similar to the preprocessing step of the coIOPP protocol of Section 5.1. The only difference now is that for the first $\log m' - c - 2$ codewords, they do not set the underlying matrixes $\mathbf{R}_i$ to be equal to $\mathbf{0}$ but, instead, they set the first two columns of the matrix $\mathbf{R}_i$ be random. Looking ahead, this minor modification will enable us to mask the sumcheck messages in the evaluation protocol. At the end of this step, each prover holds the i -th row of all codewords $\mathbf{C}_r^{(j)}$. Furthermore, it interpolates this row as a m' -degree polynomial $p_{i,r}^{(j)}$ and computes the multi-linear extension of its coefficients denoted as $g_{i,r}^{(j)}$.

Commitment Protocol. Initially, the provers interact following the Π^{Enc} protocol so that each party $\mathcal{P}_i$ eventually receives the i -th row of the codeword of $\mathbf{X}$ denoted with $\mathbf{C}^{(1)}$. In addition, each prover $\mathcal{P}_i$, also stores the m' -degree univariate polynomial $r^{(i)}(Y)$, produced by interpolating

$\mathbf{C}_i^{(1)}$, as well as $g_i : \mathbb{F}^{\log m'} \rightarrow \mathbb{F}$ that is the multi-linear extension of the coefficients of $r^{(i)}$ (step 1). Finally, the provers send the oracle of $\mathbf{C}^{(1)}$ to the verifier. This process repeats for all masking codewords $\mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log m' - c)}$.

Protocol Description. First, observe that the authors of Basefold directly encode the coefficients of f^* . This subsequently enables them to link the reduced polynomial obtained by the final codeword with the reduced sumcheck instance. Unfortunately, it is unclear how to achieve something similar in the collaborative setting. That is because our encoding scheme, directly encodes the coefficients of g —the polynomial that interpolates $\tilde{\mathbf{X}}$ —and not f^* . To overcome this, we transform the Basefold’s sumcheck instance so that it involves the polynomial $g^* : \mathbb{F}^{\log km} \rightarrow \mathbb{F}$ and not f^* . To achieve this, we observe that:

$$\begin{aligned} y &= \sum_{i,j \in [k] \times [m]} p(z_1^i, z_2^j) \beta(\mathbf{t}_1, \mathbf{i}) \beta(\mathbf{t}_2, \mathbf{j}) \\ &= \sum_{i \in [k]} \sum_{j \in [m]} p(i, j) \beta(\mathbf{t}_1, \mathbf{i}) \beta(\mathbf{t}_2, \mathbf{j}) \\ &= \sum_{i \in [k]} \sum_{j \in [m]} \left(\sum_{a,b \in [k] \times [m]} (z_1^i)^a (z_2^j)^b c_{a,b} \right) \beta(\mathbf{t}_1, \mathbf{i}) \beta(\mathbf{t}_2, \mathbf{j}) \\ &= \sum_{a,b \in [k] \times [m]} c_{a,b} \sum_{i,j \in [k] \times [m]} z_1^{ia} z_2^{jb} \beta(\mathbf{t}_1, \mathbf{i}) \beta(\mathbf{t}_2, \mathbf{j}) \\ &= \sum_{a,b \in [k] \times [m]} c_{a,b} \left(\sum_{i \in [k]} (z_1^a)^i \beta(\mathbf{t}_1, \mathbf{i}) \right) \left(\sum_{j \in [m]} (z_2^b)^j \beta(\mathbf{t}_2, \mathbf{j}) \right) \\ &= \sum_{a,b \in [k] \times [m]} c_{a,b} \cdot (p_{\mathbf{t}_1}(z_1^a) p_{\mathbf{t}_2}(z_2^b)) \end{aligned}$$

By setting the multi-linear polynomials $f_{\mathbf{t}_1} : \mathbb{F}^{\log k} \rightarrow \mathbb{F}$, $f_{\mathbf{t}_2} : \mathbb{F}^{\log m} \rightarrow \mathbb{F}$ such that $f_{\mathbf{t}_1}(\mathbf{j}) = p_{\mathbf{t}_1}(z_1^j)$, then we can replace f^* from the original Basefold sumcheck instance with g^* as follows:

$$y = \sum_{\mathbf{x}_1, \mathbf{x}_2 \in \{0,1\}^{\log k + \log m}} g^*(\mathbf{x}_1, \mathbf{x}_2) f_{\mathbf{t}_1}(\mathbf{x}_1) f_{\mathbf{t}_2}(\mathbf{x}_2) \quad (2)$$

We note, however, that there is still one minor issue. The above sumcheck is defined over g^* , the multi-linear extension of the coefficients of p^* , but $\mathbf{C}^{(1)}$ encodes the coefficients of p —the polynomial that interpolates the masked message. Consequently, we have to replace g^* with g . This is done by padding $\mathbf{t}_1$ and $\mathbf{t}_2$ with $\log k' - \log k$ and $\log m' - \log m$ zeros respectively. We denote the padded evaluation points with $\mathbf{t}'_1, \mathbf{t}'_2$.

Construction 4 gives a detailed description of our evaluation protocol. Our protocol makes use of the $\mathcal{F}_{ip}$ functionality, that takes as input the packed secret shares of the columns of a matrix $\mathbf{R} \in \mathbb{F}^{k \times m}$ and a public vector $\mathbf{y} = (\mathbf{y}_1 \otimes \mathbf{y}_2) \in \mathbb{F}^{km}$, for $\mathbf{y}_1 \in \mathbb{F}^k$ and $\mathbf{y}_2 \in \mathbb{F}^m$ and outputs $y = \sum_{i,j} \mathbf{R}[i, j] \mathbf{y}_1[i] \mathbf{y}_2[j]$.

With that in mind, all parties interact over a modified variant of our coIOPP commit protocol of our coIOPP (for the closeness of our code) in that the provers not only aggregate and then fold the codeword $\mathbf{C}^{(i)}$ with the masked one $\mathbf{C}_r^{(i)}$, but also their sumcheck instances—since

otherwise the verifier would be able to validate the last step of the sumcheck. In more detail, for every round $j \in [\log m' - c - 1]$ this is done as follows. First, all provers setup the vector containing all the folding challenges of the verifier up until that round (i.e., $\mathbf{r} = (r_1, \dots, r_j)$) and then, invoke the functionality $\mathcal{F}_{ip}$ to compute $y_r^{(j)} = \sum_{\mathbf{x}, \mathbf{y}} f_{\mathbf{t}'_1}(\mathbf{x}) f_{\mathbf{t}'_2}(\mathbf{r}, \mathbf{y}) g_r^{(j)}(\mathbf{x}, \mathbf{y})$ (step 1.a). Finally, $\mathcal{P}_1$ sends $y_r^{(j)}$ to the verifier which replies with the aggregation challenge b_j (step 1.b). Given b_j provers aggregate the sumcheck instances of $\mathbf{C}^{(j)}$ and $\mathbf{C}_r^{(j)}$ by linearly combining the polynomials $g(\mathbf{r}, \mathbf{x})$ and $g_r^{(j)}(\mathbf{x})$, and running one sumcheck round over $y + b_j y_r^{(j)} = \sum_{\mathbf{x}, \mathbf{y}} f_{\mathbf{t}'_1}(\mathbf{x}) f_{\mathbf{t}'_2}(\mathbf{r}, \mathbf{y}) (g(\mathbf{r}, \mathbf{x}) + b_j g_r^{(j)}(\mathbf{x}))$ to compute the reply polynomial $h^{(j)}(X)$.

In the collaborative setting this is done as follows. First each prover $\mathcal{P}_i$ sets $g_i(\mathbf{r}, \mathbf{x}) \leftarrow g_i(\mathbf{r}, \mathbf{x}) + b_j g_{i,r}^{(j)}(\mathbf{x})$. Then, it runs the first round of the sumcheck instance $\sum_{\mathbf{y}} g_i(\mathbf{r}, \mathbf{y}) f_{\mathbf{t}'_2}(\mathbf{r}, \mathbf{y})$ and sends the reply polynomial $\hat{h}^{(j)}(\omega_i^{(j)}, X)$ to $\mathcal{P}_1$ (step 1.c). The latter, interpolates all such polynomials into $h(Y, X)$ that has individual degree of k' and 3. Finally, it computes sumcheck reply which is set to $h^{(j)}(X) \leftarrow \sum_{\mathbf{y} \in \{0,1\}^{\log k}} \beta(\mathbf{t}_1, \mathbf{y}) h(\mathbf{y}, X)$ (step 1.d). Upon receiving $h^{(j)}(X)$, the verifier sends the folding challenge r_j to all provers which use it to fold $\mathbf{C}^{(j)} + b_j \mathbf{C}_r^{(j)}$ into $\mathbf{C}^{(j+1)}$, the oracle of which is sent to the verifier.

After completing all iterations of step 1, each prover $\mathcal{P}_i$ sends to the verifier the i -th row of $\mathbf{C}^{(\log m' - c)}$. Finally, the verifier runs if the Query algorithm of our coIOPP and the verification algorithm of the sumcheck protocol. After completing the last one, it will end up with an evaluation claim $y^{(\log m' - c)} = h^{(\log m' - c)}(r_{\log m' - c})$ which, similarly to Basefold, is validated locally by the verifier using the polynomial that interpolates the codeword $\mathbf{C}^{(\log m' - c)}$.

Optimization 2. Observe in order to perform the third check of step 3, the verifier has to compute $f_{\mathbf{t}_2}(\mathbf{r}, \mathbf{y})$, which requires $\mathcal{O}(m)$ field operations. In order to reduce verification time to $\mathcal{O}(\log m)$, the verifier delegates that step to $\mathcal{P}_1$ which uses a protocol similar to the one presented in Figure 1 of [10]. In this way, the verification time is reduced to $\mathcal{O}(\log^2 m)$ and adds an overhead of $\mathcal{O}(m)$ to $\mathcal{P}_1$ (when using the specialized sumcheck for iFFTs/FFTs presented in [11]).

Lemma 4 (Performance). *Let $L = km$ be the total size of the message and N the number of parties. Then, the coIOPP of Construction 4 has the following performance characteristics: 2*

- $k_V(L, N) \in \mathcal{O}(\log L)$
- $k_P(L, N) \in \mathcal{O}(\log L)$
- $l(L) \in \mathcal{O}(L)$ and $l_i(L, N) \in \mathcal{O}(\frac{L}{N})$
- $q(L, N) = vt(L, N) \in \mathcal{O}(l \log L + N)$
- $pt_i(L, N) \in \mathcal{O}(\frac{L}{N} \log N)$
- $cm(L, N) = Nm \log |\mathbb{F}|$ bits.

4.3.2. Proof of Theorem 3.

Construction 4: Our coIOPP for $\mathcal{R}'_{MLE}$. Assume N provers $\{\mathcal{P}_1, \dots, \mathcal{P}_N\}$ and a verifier $\mathcal{V}$. Each prover holds its own private input $\mathbf{X}_i$, and collaborates with all other provers to instantiate a IOPP for $\mathcal{R}'_{MLE}$ over the multi-linear extension $f^* : \mathbb{F}^{\log km} \rightarrow \mathbb{F}$ of the concatenation of all inputs $\mathbf{X} = (\mathbf{X}_1, \dots, \mathbf{X}_N)$.

Pre-Processing. All parties run the $\mathcal{F}_{rand}^{Enc}$ functionality to initialize and encode $\lceil \log m' - c - 1 \rceil$ (secret) random matrixes $\mathbf{R}_i \in \mathbb{F}^{k \times \frac{m}{2^{i-1}}}$ defined as follows: For all matrixes except from $\mathbf{R}_i^{\log m' - c - 1}$ all columns are zero except from the first two. As for $\mathbf{R}_i^{\log m' - c - 1}$ all columns are random. At the end of this protocol, each prover $\mathcal{P}_i$ holds the i -th row $(t, 3l)$ -zero knowledge codes $\mathbf{C}_i^{(j)}$. It also holds $g_{i,r}^{(j)}$, the multi-linear extension of the coefficients of the polynomial $p_r^{(j)}(\omega_1^{i+k'}, x)$ (which are directly obtained by $\mathbf{C}_i^{(j)}$).

Protocol Π^{Commit} . Each prover $\mathcal{P}_i$ initially has its own private input $\mathbf{X}_i \in \mathbb{F}^{k \times \frac{m}{N}}$. All provers collaborate to output a commitment of the multi-linear polynomial $f^* : \mathbb{F}^{\log km} \rightarrow \mathbb{F}$ using the following protocol:

- 1) $\mathcal{P}_i$: Invoke the Π^{Enc} protocol to obtain $\mathbf{C}_i^{(1)}$, the i -th row of $\mathbf{C}^{(1)} \in \text{RS}_1^{\otimes 2}$. In addition, form the polynomial $g_i : \mathbb{F}^{\log m'} \rightarrow \mathbb{F}$ that is the multi-linear extension of the coefficients of the m' -degree polynomial $r^{(i)}(Y)$ (obtained at step 3 of **Construction 2**).
- 2) $\mathcal{P}_1$: Send the oracle of $\mathbf{C}^{(1)}$ and those of the masking codewords $\mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log m' - c - 1)}$.

Protocol Π^{Eval} . In this protocol, all parties are given an oracle of $\mathbf{C}$, an evaluation point $\mathbf{t} = (\mathbf{t}_1, \mathbf{t}_2) \in \mathbb{F}^{\log k} \times \mathbb{F}^{\log m}$ as well as a point $y \in \mathbb{F}$ claimed to be $y = f^*(\mathbf{t})$. Furthermore, let $\mathbf{t}' = (\mathbf{t}'_1, \mathbf{t}'_2)$ be the padded evaluation points. Each prover $\mathcal{P}_i$ also holds the i -th row of $\mathbf{C}^{(1)}$, the multi-linear polynomial $g_i : \mathbb{F}^{\log m'} \rightarrow \mathbb{F}$ as well as the i -th row of the mask codewords $\mathbf{C}_r^{(j)}$ and the multi-linear extensions $g_{i,r}^{(j)} : \mathbb{F}^{\log m' - i} \rightarrow \mathbb{F}$ as defined in the main body. All parties interact as follows:

- 1) For $j = \{1, \dots, \log m' - c - 1\}$ work as follows:
 - a) $\mathcal{P}_1, \dots, \mathcal{P}_N$: If $j = 1$ set $\mathbf{r} \leftarrow \{\}$ otherwise $\mathbf{r} \leftarrow \{r_1, \dots, r_{j-1}\}$. Call the $\mathcal{F}_{ip}$ functionality to compute the inner product $y_r^{(j)} = \sum_{\mathbf{x}, \mathbf{y}} f_{t'_1}(\mathbf{x}) f_{t'_2}(\mathbf{r}, \mathbf{y}) g_r^{(j)}(\mathbf{x}, \mathbf{y})$.
 - b) $\mathcal{V}$: Samples a random point $b_j \in \mathbb{F}$ and sends it to the provers.
 - c) $\mathcal{P}_i$: Set $g_i(\mathbf{r}, \mathbf{x}) \leftarrow g_i(\mathbf{r}, \mathbf{x}) + b_j g_{i,r}^{(j)}(\mathbf{x})$. Then, compute and send to $\mathcal{P}_1$ the polynomial $\hat{h}^{(j)}(\omega_1^i, X) = \sum_{\mathbf{x} \in \{0,1\}^{\log m' - j}} g_i(\mathbf{r}, X, \mathbf{x}) f_{t'_2}(\mathbf{r}, X, \mathbf{x})$.
 - d) $\mathcal{P}_1$: Collect all $\hat{h}^{(j)}(\omega_1^1, X), \dots, \hat{h}^{(j)}(\omega_1^N, X)$, interpolate the polynomial $h(Y, X)$ of individual degree k' and 3 respectively. Finally, send $h^{(j)}(X) \leftarrow \sum_{\mathbf{y} \in \{0,1\}^k} \beta(\mathbf{t}_1, \mathbf{y}) h(\mathbf{y}, X)$ to $\mathcal{V}$.
 - e) $\mathcal{V}$: Pick a random point r_j and send it to all the provers.
 - f) $\mathcal{P}_i$: Set $\mathbf{C}_i^{(j+1)} \leftarrow \text{Fold}_{RS}(r_j, \mathbf{C}_i^{(j)} + b_j \mathbf{C}_{r,i}^{(j)})$. $\mathcal{P}_1$ sends the accumulated oracle $\mathbf{C}^{(j+1)}$ to $\mathcal{V}$.
- 2) $\mathcal{P}_i$: Send $\mathbf{C}_i^{(\log m' - c)}$ directly to $\mathcal{V}$.
- 3) $\mathcal{V}$: Check if: (1) *coIOPP.Query* outputs accept, (2) $h^{(i-1)}(r_{i-1}) + b_i y_r^{(i)} = h^{(i)}(0) + h^{(i)}(1)$ with $h^{(0)}(r_0) = y$ and (3) $h^{(\log m' - c - 1)}(r_{\log m' - c}) = \sum_{\mathbf{x}, \mathbf{y} \in \{0,1\}^{\log k' + \log c}} f_{t'_1}(\mathbf{x}) f_{t'_2}(\mathbf{r}, \mathbf{y}) g'(\mathbf{x}, \mathbf{y})$, where g' the multi-linear polynomial obtained by decoding $\mathbf{C}^{(\log m' - c)}$ and $\mathbf{r} = (r_1, \dots, r_{\log m' - c - 1})$.

Lemma 5 (Completeness). *The coIOPP of Construction 4 has perfect completeness.*

Proof. We show that if the provers follow honestly the protocol and if $f(\mathbf{t}') = y$, then the verifier will always accept. From the completeness of our coIOPP we know that check (1) will pass with probability 1. Furthermore, because of the completeness of the sumcheck protocol, checks of (2) will also pass with probability 1. Finally, for check (3), due to the coIOPP completeness we have that:

$$g'(\mathbf{x}, \mathbf{y}) = g^{(\log m' - c - 1)}(r_{\log m' - c - 1}, \mathbf{x}, \mathbf{y}) +$$

$$b_{r_{\log m' - c - 1}} g_r^{(\log m' - c - 1)}(r_{\log m' - c - 1}, \mathbf{x}, \mathbf{y})$$

and because of the completeness of the sumcheck protocol it holds that:

$$h^{(f)}(r_f) = \sum_{\mathbf{x}, \mathbf{y}} f_{t'_1}(\mathbf{x}) f_{t'_2}(\mathbf{r}, \mathbf{y}) \left(g^{(f)}(r_f, \mathbf{x}, \mathbf{y}) + b_{r_f} g_r^{(f)}(r_f, \mathbf{x}, \mathbf{y}) \right) = \sum_{\mathbf{x}, \mathbf{y}} f_{t'_1}(\mathbf{x}) f_{t'_2}(\mathbf{r}, \mathbf{y}) g'(\mathbf{x}, \mathbf{y})$$

where $f = \log m' - c - 1$. $\square$

Lemma 6 (t -zero knowledge). *Construction 4 is a perfect honest-verifier and prover t -zero knowledge coIOPP in the $\mathcal{F}_{rand}^{Enc}, \mathcal{F}_{ip}$ -Hybrid model with a corrupted prover threshold of $t \leq N - k + 1$.*

Proof. We first construct a simulator $\mathcal{S}$ that has access to the random tape of the adversary that corrupts at most t provers

$\{\mathcal{P}_{j_1}, \dots, \mathcal{P}_{j_t}\}$ and the verifier $\mathcal{V}$ as well as the inputs of the corrupted provers. Our simulator works as follows:

- 1) First, it invokes step 1 of $\mathcal{S}^{coIOPP}$ —the simulator of the coIOPP protocol—to obtain all replies from the coIOPP query phase, as well as the polynomial g' . Then, it samples the bivariate polynomials $\hat{h}^{(1)}(Y, X), \dots, \hat{h}^{(\log m' - c - 1)}(Y, X)$ (computed at step 1.d) uniformly at random with the only restriction that $\hat{h}^{(i)}(\omega_1^j, X)$, for all $j \in \mathcal{J}, i \in [\log m' - c - 1]$ are consistent with what the corrupted provers compute. Next, it instantiates the simulators $\mathcal{S}_1$ and $\mathcal{S}_2$ of the commitment and evaluation phase.
- 2) **Simulator $\mathcal{S}_1$.** The simulator runs step 2 and the beginning of step 3 of $\mathcal{S}^{coIOPP}$ (that is, up to sending the oracle of $\mathbf{C}^{(1)}$).
- 3) **Simulator $\mathcal{S}_2$.** Next, the simulator runs the rest of $\mathcal{S}^{coIOPP}$ for the evaluation phase. In addition to that, however, for each round i , it also simulates steps 1.a – 1.c as follows. In the first iteration, it sets $y_r^{(1)} = b_1^{-1}(h^{(1)}(0) + h^{(1)}(1) - y)$ while in the rest of the rounds it sets $y_r^{(j)} = b_j^{-1}(h^{(j)}(0) + h^{(j)}(1) - h^{(j-1)}(r_{j-1}))$ and simulates the functionality $\mathcal{F}_{ip}$ by returning the point $y_r^{(i)}$. Then, it locally computes the polynomials $\hat{h}^{(i)}(\omega_1^j, X), \dots, \hat{h}^{(i)}(\omega_1^{j_t}, X)$ and samples the rest of them uniformly at random such that they all evaluate at $\hat{h}^{(i)}(Y, X)$. These polynomials and also $h^{(i)}$ are sent to the adversary.
- 4) Let $V = (V_1, V_2, V_3)$ be the simulated view such that (1) V_1 contains the oracles, (2) V_2 the polynomials $\hat{h}^{(1)}(Y, X), \dots, \hat{h}^{(\log m' - c - 1)}(Y, X)$ (we omit the sumcheck polynomials, the univariate polynomials of the form $\hat{h}^{(i)}(\omega_1^j, X)$ and the evaluations $y_r^{(1)}, \dots, y_r^{(\log m' - c - 1)}$ since they can be derived by those polynomials), and (3) V_3 the data returned by the simulators during the Π^{Enc} protocol and the replies to the queries of the *Query* phase of the coIOPP protocol, that is, the view of the coIOPP protocol.

Let V^* be the view of the adversary when running the PCS on the real world. We will show that the distributions of (V_1, V_2, V_3) and (V_1^*, V_2^*, V_3^*) are identical. Indistinguishability between V_1 and V_1^* stems directly from the indistinguishability of the oracles. Hence, we will focus on proving $(V_2, V_3) \equiv (V_2^*, V_3^*)$. For this, we use an almost identical process with the one of Lemma 2. Namely, we will show that given any such view, and any original input $\mathbf{X}$, we can generate the polynomials $p, p_r^{(1)}, \dots, p_r^{(\log m' - c)} : \mathbb{F}^2 \rightarrow \mathbb{F}$ that produce $\mathbf{C}^{(1)}, \mathbf{C}_r^{(1)}, \dots, \mathbf{C}_r^{(\log m' - c)}$, and that the number of these polynomials is constant for any different accepting view V_2 .

With that in mind, we begin by first showing how to find the polynomials p and $p_r^{(1)}$. Note that we can not interpolate $p_r^{(1)}$ by directly following the proof of Lemma 2. This is because $\mathbf{R}_1$ is not zero and, in fact, need to evaluate it in such a way that is consistent with the sumcheck reply $h^{(1)}(X)$. To achieve this, we first compute the polynomial $h_r^{(1)}(Y, X) = b_1^{-1}(\hat{h}^{(1)}(Y, X) - h_x(Y, X))$,

where $h_x(Y, X)$ and $h_r^{(1)}(Y, X)$ are bivariate polynomials such that $h_x(\omega^i, X) = \sum_{\mathbf{x} \in \{0,1\}^{\log m' - 1}} f_{t_2'}(X, \mathbf{x}) g_i(X, \mathbf{x})$ and $h_r(\omega^i, X) = \sum_{\mathbf{x} \in \{0,1\}^{\log m' - 1}} f_{t_2'}(X, \mathbf{x}) g_{i,r}^{(1)}(X, \mathbf{x})$ respectively. At this point, recall that only the first two columns of $\mathbf{R}_1$ are non-zero, which implies that only the first two coefficients $g_{i,r}^{(1)}$ are non-zero (that is, $g_{i,r}^{(1)}(0, \mathbf{0})$ and $g_{i,r}^{(1)}(1, \mathbf{0})$). Thus, we compute $g_{i,r}^{(1)}$ by setting $g_{i,r}(0) = h_r(\omega_1^i, 0)$ and $g_{i,r}(1) = h_r(\omega_1^i, 1)$ and use them to derive the columns of $\mathbf{R}_1$. Given $\mathbf{R}_1$, we can now follow the same process as with Lemma 2 to compute $p_r^{(1)}$. Finally, we recursively continue following the same process to compute all the other polynomials $p_r^{(2)}, \dots, p_r^{(\log m' - c - 1)}$.

Up until now, we have shown that for any $\mathbf{X}$ and any accepting view V_2 , we can find a set of polynomials $p, p_r^{(1)}, \dots, p_r^{(\log m' - c - 1)}$ that produce V_2 . It remains to show that the number of these polynomials is constant for any V_2 . For this we follow the same argument as in Lemma 2. $\square$

Lemma 7 ((Round-By-Round) Soundness). *Let $\pi^{(1)}, \pi_r^{(1)}, \pi_r^{(2)}, \dots, \pi_r^{(\log m' - c)}$ be the messages of the oracles sent to the verifier in the first step of the commitment algorithm. In addition, let $\delta = \min(\max(\Delta(\pi^{(1)}, \mathbf{C}^{(1)}), \{\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)})\}_{i \in [\log m' - c - 1]}), \frac{\Delta}{3})$ where $\mathbf{C}^{(i)} \in RS_i^{\otimes 2}$. Assuming that $(1 - \delta)^l < \text{negl}(\lambda)$ and that $\frac{|\pi^{(1)}|}{|\mathbb{F}|} < \text{negl}(\lambda)$, then, if the verifier of **Construction 4** passes the checks of step 3, with probability $1 - \text{negl}(\lambda)$, it holds that $\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)}) \leq \delta$, for all $i \in [\log m' - c - 1]$ and there exists a unique polynomial $f : \mathbb{F}^{\log k' m'} \rightarrow \mathbb{F}$ such that $\Delta(\pi_r^{(1)}, \text{Enc}(f)) \leq \delta$ and $f(\mathbf{t}') = y$.*

Proof. To prove the above lemma, we first show that our IOPP has negligible round-by-round soundness error. Then using Proposition 5.4 of [12] we upper bound standard soundness error. Due to space limitations, we proved a detailed proof of the round-by-round soundness of **Construction 5** in Appendix 4. $\square$

Lemma 8 ((Round-By-Round) Knowledge Soundness). *Let $\delta < \frac{\Delta}{3}$ and l that satisfy conditions of Lemma 7. Then, for any instance $(\pi^{(1)}, \pi_r^{(1)}, \dots, \pi_r^{(\log m' - c - 1)}, y, \mathbf{t})$ and any PPT prover that generates an accepting proof, there exists a PPT extractor that runs in (expected) polynomial time and outputs a multi-linear polynomial $f : \mathbb{F}^{\log k' m'} \rightarrow \mathbb{F}$ such that $f(\mathbf{t}) = y$ and $\Delta(\pi^{(1)}, \text{Enc}(f)) < \delta$ with probability of $1 - \kappa > 1 - \text{negl}(\lambda)$. Last but not least, our scheme has round-by-round knowledge error $\kappa_{rbr} = \max(\frac{|\mathbf{C}^{(1)}|}{|\mathbb{F}|}, (1 - \delta)^l)$.*

Proof. We can create an extractor by following the same steps as in Theorem 4 of [9]. The success probability of the underlying extractor is guaranteed by Lemma 7. Note that this extractor runs in expected polynomial time. Because our collaborative code is efficiently decodable, and $\delta < \frac{\Delta}{3}$, it is also possible to directly use a straightline extractor as proposed in [13]. Finally, in Appendix 4 we show that

our construction has negligible round-by-round knowledge soundness error. $\square$

4.3.3. Proof of Lemma 7.

Proof. Let $\delta < \frac{\Delta}{3}$. $\mathbf{t}_0 = (\pi^{(1)}, \{\pi_r^{(i)}\}_{i \in [\log m' - c] - 1}, y, \mathbf{r})$ be the initial state. We say that the initial state is doomed if and only if one of the following conditions hold:

- 1) $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) > \delta$ or $\exists i \in [\log m' - c - 1]$ such that $\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)}) > \delta$.
- 2) $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) \leq \delta$ but $y \neq f(\mathbf{t}') = \sum_{\mathbf{x}, \mathbf{y}} g(\mathbf{y}, \mathbf{x}) f_{\mathbf{t}'_1}(\mathbf{y}) f_{\mathbf{t}'_2}(\mathbf{x})$, where $f : \mathbb{F}^{\log k' m'} \rightarrow \mathbb{F}$ the multi-linear extension of the unique (since $\delta < \frac{\Delta}{3}$) message obtained by $\pi^{(1)}$.

Next, for each of the $j \in [\log m' - c - 1]$ rounds of the evaluation protocol, we define the following two states and describe the conditions on which they belong to the doomed set $\mathcal{D}$:

- The state $\mathbf{t}_{2i} = (\mathbf{t}_{2i-1}, \pi^{(j)}, y_r^{(j)}, b_j)$. We say that this state is doomed if for a randomly chosen b_j one of the following hold:
 - 1) There exists $i > j$, such that $\pi_r^{(i)}$, where $\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)}) > \delta$.
 - 2) There exists $i \in \{1, \dots, j\}$ such that $\pi^{(i)} \neq \text{Fold}(r_j, \pi^{(i-1)} + b_{i-1}\pi_r^{(i-1)})$.
 - 3) $\Delta(\pi^{(j)} + b_j\pi_r^{(j)}, \mathbf{C}^{(j)}) > \delta$.
 - 4) $\Delta(\pi^{(j)} + b_j\pi_r^{(j)}, \mathbf{C}^{(j)}) \leq \delta$, but $y_i + ay_r^{(j)} \neq \sum_{\mathbf{x}} (g^{(j)}(\mathbf{x}) + bg_r^{(j)}(\mathbf{x})) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$, where $g^{(j)}, g_r^{(j)}$ the unique polynomials obtained by $\pi^{(j)}$ and $\pi_r^{(j)}$.
- The state $\mathbf{t}_{2i+1} = (\mathbf{t}_{2i}, h^j(X), r_j)$. This state is doomed if for a randomly chosen r_j the following holds:
 - 1) There exists $i > j$, such that $\pi_r^{(i)}$, where $\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)}) > \delta$.
 - 2) There exists $i \in \{1, \dots, j\}$ such that $\pi^{(i)} \neq \text{Fold}(r_j, \pi^{(i-1)} + b_{i-1}\pi_r^{(i-1)})$.
 - 3) $\pi^{(j+1)} = \Delta(\text{Fold}(r_j, \pi^{(j)} + b_j\pi_r^{(j)}), \mathbf{C}^{(j+1)}) > \delta$
 - 4) $\Delta(\text{Fold}(r_j, \pi^{(j)} + b_j\pi_r^{(j)}), \mathbf{C}^{(j+1)}) \leq \delta$ but $y^{(j)} = h^{(j)}(r_j) \neq \sum_{\mathbf{x}} g^{(j+1)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, r_j, \mathbf{x})$, where $g^{(j+1)}(\mathbf{x}) \leftarrow g^{(j)}(r_j, \mathbf{x}) + b_j g_r^{(j)}(r_j, \mathbf{x})$.

Provided these states, we will first find the probabilities $\epsilon_{2j}, \epsilon_{2j+1}$ where $\mathbf{t}_{2j}$ and $\mathbf{t}_{2j+1}$ transition from a doomed to a non-doomed state. For this, we will show that:

$$\epsilon_{2j} = \Pr_{b_j \xleftarrow{\mathbb{F}}} [\mathbf{t}_{2j} \notin \mathcal{D}] \leq \frac{|\pi^{(j)}|}{|\mathbb{F}|} + \frac{1}{|\mathbb{F}|} \text{ and } \epsilon_{2j+1} =$$

$$\Pr_{r_j \xleftarrow{\mathbb{F}}} [\mathbf{t}_{2j+1} \notin \mathcal{D}] \leq \frac{|\pi^{(j)}|}{|\mathbb{F}|} + \frac{2}{|\mathbb{F}|}$$

We will prove the above inductively starting from $j = 1$ and assuming that $\mathbf{t}_0 \in \mathcal{D}$. First, we find ϵ_{2j} and show that for any randomly sampled $b_j \in \mathbb{F}$, the probability $(\mathbf{t}_0, \pi^{(1)}, y_r^{(1)}) \notin \mathcal{D}$ is negligible. We consider three cases:

- 1) E_1 : In case $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) > \delta$, then due to the Lemma 1, it will hold that $\Delta(\pi^{(1)} + b_1\pi_r^{(1)}) \leq \delta$ only with probability $\frac{|\pi^{(1)}|}{|\mathbb{F}|}$.

- 2) E_2 : In case $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) \leq \delta$ and $y \neq \sum_{\mathbf{x}} g(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{x})$, if $\Delta(\pi^{(1)} + b_1\pi_r^{(1)}) > \delta$, then the prover stays in a doomed state with probability 1. Hence, we assume that $\Delta(\pi^{(1)} + b_1\pi_r^{(1)}) < \delta$. Since $y = y^{(1)} \neq f(\mathbf{t}')$ then by the Schwartz-Zippel lemma, the prover leaves the doomed state with probability of only $\frac{1}{|\mathbb{F}|}$.

- 3) E_3 : There is $\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)}) > \delta$ for $i > 1$. In this case, the prover moves to a doomed state with probability 1.

By taking the union bound of both cases, we get that $\Pr[(\mathbf{t}_0, \pi^{(1)}, y_r^{(1)}, b_1) \notin \mathcal{D}] \leq \frac{|\pi^{(1)}|}{|\mathbb{F}|} + \frac{1}{|\mathbb{F}|}$. Next, assuming that $(\mathbf{t}_0, \mathbf{t}_1)$ are in a doomed state, we will show that also $(\mathbf{t}_0, \mathbf{t}_1, h^{(1)}(X))$ will remain in a doomed state with overwhelming probability. Once again, we consider three cases:

- 1) E'_1 : From the previous doomed state it holds that $\Delta(\pi^{(1)} + b_1\pi_r^{(1)}) > \delta$. Then, by Lemma 3, it holds that $\pi^{(2)} = \Delta(\text{Fold}(r_1, \Delta(\pi^{(1)} + b_1\pi_r^{(1)})), \mathbf{C}^{(2)}) < \epsilon$ only with probability $\frac{|\pi^{(1)}|}{|\mathbb{F}|}$.
- 2) E'_2 : From the previous doomed state it holds that $\Delta(\pi^{(1)} + b_1\pi_r^{(1)}, \mathbf{C}^{(2)}) \leq \delta$ but $y^{(1)} + b_1y_r^{(1)} \neq \sum_{\mathbf{x}} (g^{(1)}(\mathbf{x}) + b_1g_r^{(1)}(\mathbf{x})) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{x})$. In this case, $\text{Fold}(r_1, \pi^{(1)} + b_1\pi_r^{(1)}) \leq \delta$ and $g^{(2)}(\mathbf{x}) = g^{(1)}(r_1, \mathbf{x}) + b_1g_r^{(1)}(r_1, \mathbf{x})$, where $g^{(2)}$ the unique polynomial obtained by the folded codeword. Due to the Schwartz-Zippel lemma, $h^{(1)}(r_1) = y^{(2)}$ equals to $\sum_{\mathbf{x}} g^{(2)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(r_1, \mathbf{x}) = \sum_{\mathbf{x}} (g^{(1)}(r_1, \mathbf{x}) + b_1g_r^{(1)}(r_1, \mathbf{x})) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(r_1, \mathbf{x})$ with probability of only $\frac{2}{|\mathbb{F}|}$.
- 3) E_3 : There is $\Delta(\pi_r^{(i)}, \mathbf{C}^{(i)}) > \delta$ for $i > 1$. In this case, the prover moves to a doomed state with probability 1.

We follow an identical argument for any $j \in \{2, \dots, \log m' - c\}$, but we also consider one additional case, each time, where there exists $i \in \{1, \dots, j\}$ such that $\pi^{(i)} \neq \text{Fold}(r_j, \pi^{(i-1)} + b_{i-1}\pi_r^{(i-1)})$ and either $\Delta(\pi^{(i-1)} + b_{i-1}\pi_r^{(i-1)}, \mathbf{C}^{(i-1)}) > \delta$ or $g^{(i)} \neq g^{(i)}$ such that $\Delta(\pi^{(i)}, \text{Enc}(g^{(i)})) \leq \delta$. If this case is true, then the prover stays to a doomed state with probability 1.

Finally, we define the transcript $\mathbf{t}_f = (\mathbf{t}_{2(\log m' - c)}, \mathbf{C}^{(\log m' - c)}, \mathcal{I} = \{(x_1, y_1), \dots, (x_l, y_l)\})$, where $\mathbf{C}^{(\log m' - c)}$ the last codeword sent by the prover, and $\mathcal{I}$ the coIOPP queries of the verifier. We say that $\mathbf{t}_f$ is doomed if the following hold:

- 1) $\mathbf{C}^{(\log m' - c)}$ is not a valid codeword.
- 2) $h^{(\log m' - c - 1)}(r_{\log m' - c - 1}) \neq \sum_{\mathbf{x}} g^{(\log m' - c)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$, where $g^{(\log m' - c)}$ the polynomial obtained by $\mathbf{C}^{(\log m' - c)}$.
- 3) There is an inconsistent query in the *coIOPP.Query* algorithm.

Assume that $\mathbf{t}_{2(\log m' - c) - 1} \in \mathcal{D}$, then, we will show that the probability $\mathbf{t}_f \notin \mathcal{D}$ is at most $(1 - \delta)^l$. Observe that if $\mathbf{C}^{(\log m' - c)}$ is not a valid codeword, the verifier rejects with probability 1. Next, assume that $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) > \delta$, or $\exists \pi^{(i)}$

such that $\Delta(\pi^{(i)}, \mathbf{C}^{(i)}) > \delta$ or $\Delta(\pi^{(i)} + b_i \pi_r^{(i)}, \mathbf{C}^{(i)}) > \delta$. By Theorem 2, we get that the probability the verifier passes the *coIOPP.Query* is $(1 - \delta)^l$. Finally, assume that $\Delta(\pi^{(1)}, \mathbf{C}^{(1)}) < \delta$ but $f(\mathbf{t}') \neq y$. We furthermore, assume that $\forall \pi^{(i)}$, it holds that $\Delta(\pi^{(i)}, \mathbf{C}^{(i)}) \leq \delta$ and $\Delta(\pi^{(i)} + b_i \pi_r^{(i)}, \mathbf{C}^{(i)}) \leq \delta$ since otherwise the *coIOPP.Query* algorithm would fail with overwhelming probability. Then, the only way the prover can move to a non-doomed would be to send an oracle π_i that encodes a polynomial $g'^{(i)}$ different than that of $Fold(r_{i-1}, \pi^{(i-1)} + b_{i-1} \pi_r^{(i-1)})$, and also $\Delta(\pi_i, Enc(g'^{(i)})) \leq \delta$. Following the same argument as in [9], we can show that if the prover passes the *coIOPP*, then with overwhelming probability it holds that $\Delta(\pi^{(i)}, Enc(g'^{(i)})) \leq 2\delta$, and thus, we end up with $\Delta \leq \Delta(Enc(g'^{(i)}), g^{(i)}) \leq 3\delta < \Delta$ which is a contradiction. $\square$

4.3.4. Proof of Round-by-Round Knowledge Soundness.

We prove round-by-round knowledge soundness using the same process with the one shown in [14]. We focus on the first $2(\log m' - c - 1) + 1$ states as defined in our proof for round-by-round soundness. Assume that:

$$\Pr_{r_j \leftarrow \mathbb{F}}[(\mathbf{t}_{2j}, h^{(j)}(X)) \notin \mathcal{D}] > \epsilon_{rbr}$$

Then, we will show that $\mathbf{t}_{2j} \notin \mathcal{D}$. Since $(\mathbf{t}_{2j}, h^{(j)}(X)) \notin \mathcal{D}$, it implies that $\pi^{(i)} = Fold(r_i, \pi^{(i-1)} + b_{i-1} \pi_r^{(i-1)})$. Consequently, if $\mathbf{t}_{2j} \in \mathcal{D}$, one of the following should hold:

- 1) $\Delta(\pi^{(j)} + b_j \pi_r^{(j)}, \mathbf{C}^{(j)}) > \delta$ but $\Delta(Fold(r_j, \pi^{(j)} + b_j \pi_r^{(j)}), \mathbf{C}^{(j+1)}) \leq \delta$
- 2) $\Delta(Fold(r_j, \pi^{(j)} + b_j \pi_r^{(j)}), \mathbf{C}^{(j+1)}) \leq \delta$ and $y^{(j)} + ay_r^{(j)} \neq \sum_{\mathbf{x}} (g^{(j)}(\mathbf{x}) + b_j g_r^{(j)}(\mathbf{x})) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$ but $y^{(j)} = \sum_{\mathbf{x}} g^{(j+1)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, r_j, \mathbf{x})$

Observe that the first case can happen with probability larger than ϵ_{rbr} if and only if $\Delta(\pi^{(j)} + b_j \pi_r^{(j)}, \mathbf{C}^{(j)}) \leq \delta$, since otherwise Lemma 3 would be invalid, leading to a contradiction. Furthermore, due to the Schwartz-Zippel Lemma, the second case can happen with probability larger than ϵ_{rbr} only if $y^{(j)} + ay_r^{(j)} = \sum_{\mathbf{x}} (g^{(j)}(\mathbf{x}) + b_j g_r^{(j)}(\mathbf{x})) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$. Consequently, $\mathbf{t}_{2j} \notin \mathcal{D}$.

Next, assume that:

$$\Pr_{b_j \leftarrow \mathbb{F}}[(\mathbf{t}_{2j-1}, \pi^{(j)}, y_r^{(j)}) \notin \mathcal{D}] > \epsilon_{rbr}$$

We will show that $\mathbf{t}_{2j-1} \notin \mathcal{D}$ must hold. As above, $(\mathbf{t}_{2j-1}, \pi^{(j)}, y_r^{(j)}) \notin \mathcal{D}$ implies that $\pi^{(i)} = Fold(r_{i-1}, \pi^{(i-1)} + b_{i-1} \pi_r^{(i-1)})$ for all $i \in [j]$. In addition, if $\mathbf{t}_{2j-1} \in \mathcal{D}$, the following must hold:

- 1) $\Delta(\pi^{(j)}, \mathbf{C}^{(j)}) > \delta$ or $\Delta(\pi_r^{(j)}, \mathbf{C}^{(j)}) > \delta$ and $\Delta(\pi^{(j)} + b_j \pi_r^{(j)}, \mathbf{C}^{(j)}) \leq \delta$.
- 2) $\Delta(\pi^{(j)}, \mathbf{C}^{(j)}) \leq \delta$, $\Delta(\pi_r^{(j)}, \mathbf{C}^{(j)}) \leq \delta$, and $y^{(j)} \neq \sum_{\mathbf{x}} g^{(j)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$ or $y_r^{(j)} \neq \sum_{\mathbf{x}} g_r^{(j)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$ but $y^{(j)} + ay_r^{(j)} = \sum_{\mathbf{x}} (g^{(j)}(\mathbf{x}) + b_j g_r^{(j)}(\mathbf{x})) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$.

It is not hard to see that if the first case is true for probability larger than ϵ_{rbr} , then it contradicts Lemma A.1 of [2]. Likewise, due to the Schwartz-Zippel Lemma, the second case happens with probability larger than ϵ_{rbr} if and only if y_r and $y^{(j)}$ are correct. Hence, we conclude that $\mathbf{t}_{2j-1} \notin \mathcal{D}$.

We follow the same process inductively to show that $\mathbf{t}_0 \notin \mathcal{D}$. What remains is to show that if $\mathbf{t}_f = (\mathbf{t}_{2(\log m' - c - 1)}, \pi^{(\log m' - c)}, \mathcal{I}) \notin \mathcal{D}$ with probability larger than ϵ_{rbr} , then it must hold that $\mathbf{t}_{2(\log m' - c - 1)} \notin \mathcal{D}$. Since $\mathbf{t}_{2(\log m' - c)} \in \mathcal{D}$, we consider three cases:

- 1) $\Delta(\pi^{(\log m' - c)}, \mathbf{C}^{(\log m' - c)}) > \delta$
- 2) $\Delta(\pi^{(\log m' - c)}, \mathbf{C}^{(\log m' - c)}) = 0$
but $h^{(\log m' - c - 1)}(r_{\log m' - c - 1}) \neq \sum_{\mathbf{x}} g^{(\log m' - c)}(\mathbf{x}) f_{\mathbf{t}'_1 \otimes \mathbf{t}'_2}(\mathbf{r}, \mathbf{x})$.
- 3) There exists $i \in [\log m' - c - 1]$ such that $\pi^{(i)} \neq Fold(r_i, \pi^{(i-1)} + b_i \pi_r^{(i-1)})$.

Since the verification algorithm is accepting, the first and second case must not hold. Furthermore, by the soundness analysis of the query phase, and by the assumption that $\mathbf{t}_0 \in \mathcal{D}$, the third case only holds with probability of at most $(1 - \delta)^l$. Consequently, $\mathbf{t}_{2(\log m' - c - 1)} \notin \mathcal{D}$.

5. Proofs of Section 6

5.1. Proofs for our coSumcheck Protocol

Theorem 4. Fix a parameter $\delta < \frac{\Delta}{3}$. **Construction 5** is a coPIOP for proving the sumcheck instance $K = \sum_{\mathbf{x} \in \{0,1\}^{\log n}} f(\mathbf{x})$ that is perfectly complete and honest verifier and prover t -zero knowledge in the $\mathcal{F}_{rand}^{Enc}$, $\mathcal{F}_{sumcheck}$, $\mathcal{F}_{ip}$ -hybrid model. Furthermore, it has soundness error of $\epsilon = \frac{1 + \log n}{|\mathbb{F}|} + (1 - \delta)^l$ and round-by-round (knowledge) soundness errors of $\epsilon_{rbr} = \kappa_{rbr} = \max(\frac{2 \log n}{|\mathbb{F}|}, (1 - \delta)^l)$.

5.1.1. Completeness and Performance Analysis.. Completeness comes from the completeness of $\mathcal{F}_{rand}^{Enc}$, the collaborative sumcheck protocol of [16], [15] as well as the homomorphic properties of our collaborative code. Performance-wise, our coSumcheck protocol has the following properties. First, $k_V(n, N)$ and $k_P(n, N)$ are $\mathcal{O}(\log n)$. The first one, is due to step 3, the sumcheck rounds and steps 3, 5. The complexity of $k_P(n, N)$ derives from steps 2, 4 and 5.a. In addition, verifier has to make a single query over the polynomial oracle of f and $\mathcal{O}(l \log(l + \log n) + N)$ queries over $\mathbf{C}^{(1)}$ and $\mathbf{C}^{(2)}$ (when optimization 3 is applied). Likewise, we argue about verification time which is $vt(n, N) \in \mathcal{O}((l + \log n)N)$.

It is not hard to see that $pt_i(n, N) \in \mathcal{O}(\frac{n}{N})$ since, the prover time is dominated by step 4 which requires $\mathcal{O}(\frac{n}{N})$ field operations [15]. Last but not least, note that $cm(n, N) \in \mathcal{O}(\log n)$ because of the steps 2, 4 and 5.a.

5.1.2. Proof of t -zero knowledge.. We begin by constructing a simulator $\mathcal{S}$ that has access to an adversary that corrupts t provers with indexes $\mathcal{J} = \{j_1, \dots, j_t\}$ and the

Construction 5: Our coSumcheck. Each prover $\mathcal{P}_i$ is given a packed secret share of the coefficients of the multi-linear polynomial $f : \mathbb{F}^{\log n} \rightarrow \mathbb{F}$, denoted with $\llbracket f \rrbracket_i$. We assume that the verifier has a polynomial oracle of f .

Pre-Processing. All provers invoke the functionality $\mathcal{F}_{rand}^{Enc}$ for the random matrixes $\mathbf{R}_1, \mathbf{R}_2 \in \mathbb{F}^{k \times (\log \frac{n}{k})}$. Each prover $\mathcal{P}_i$ holds the i -th rows of the codewords $\mathbf{C}^{(1)}, \mathbf{C}^{(2)}$. In addition, each prover also uses that row to retrieve the i -th packed secret share of each column of $\mathbf{R}_1, \mathbf{R}_2$. Finally, the provers send the oracles of $\mathbf{C}^{(1)}, \mathbf{C}^{(2)}$ to $\mathcal{V}$.

Protocol $\Pi^{coSumcheck}$. Each prover has a claimed sum $K \in \mathbb{F}$, and wants to convince a verifier $\mathcal{V}$ that $K = \sum_{\mathbf{x} \in \{0,1\}^{\log n}} f(\mathbf{x})$.

- 1) $\mathcal{P}_i$: Using the packed secret shares of $\mathbf{R}_1$, each prover instantiates the packed secret shares of the multi-linear polynomial $f_r : \mathbb{F}^{\log n} \rightarrow \mathbb{F}$, that is the multi-linear extension of the matrix $\mathbf{R}'_1 \in \mathbb{F}^{k \times \frac{n}{k}}$ where all columns are zero except from those with indexes $i \in \{1, 2, \dots, 2^{\log \frac{n}{k}}\}$, which equal to the $\log i$ -th row of $\mathbf{R}_1$.
- 2) $\mathcal{P}_1, \dots, \mathcal{P}_N$: Invoke the functionality $\mathcal{F}_{ip}$, with input $\mathbf{R}_1$ and public vectors filled with ones to compute $K' = \sum_{\mathbf{x} \in \{0,1\}^{\log n}} f_r(\mathbf{x})$. Finally, $\mathcal{P}_1$ sends K' .
- 3) $\mathcal{V}$: Selects a random point b and sends it to the provers.
- 4) All parties invoke the $\mathcal{F}_{sumcheck}$ functionality (which can be instantiated using the coSumcheck protocol of [15]) for the instance $K + bK' = \sum_{\mathbf{x} \in \{0,1\}^{\log n}} (f(\mathbf{x}) + bf_r(\mathbf{x}))$. All parties get a sumcheck proof as well as the evaluation claims y and y_r of f, f_r at the point $\mathbf{r}$.
- 5) $\mathcal{V}$: Validates the last round of the sumcheck using y, y_r and checks the validity of y by querying the polynomial oracle of f . For y_r it works as follows:
 - a) $\mathcal{P}_1, \dots, \mathcal{P}_N$: Invoke the $\mathcal{F}_{ip}$ functionality (using $\mathbf{R}_2$) to compute $y'_r \leftarrow f'_r(\mathbf{r})$ where $f'_r : \mathbb{F}^{\log n} \rightarrow \mathbb{F}$ be a polynomial defined similarly as f_r but using $\mathbf{R}_2$. $\mathcal{P}_1$ sends y'_r to $\mathcal{V}$.
 - b) $\mathcal{V}$: Sends $c \in \mathbb{F}$ to all provers.
 - c) $\mathcal{P}_i$: Sends $\mathbf{C}'_i \leftarrow \mathbf{C}^{(1)}_i + c\mathbf{C}^{(2)}_i$ to $\mathcal{V}$.
 - d) $\mathcal{V}$: Forms $\mathbf{C}' = (\mathbf{C}'_1, \dots, \mathbf{C}'_N)$ and performs the following checks: (1) It ensures that $\mathbf{C}'$ is a valid codeword and decodes $\mathbf{R}' = \mathbf{R}_1 + c\mathbf{R}_2$, (2) makes l queries over the oracles of $\mathbf{C}^{(1)}, \mathbf{C}^{(2)}$ and checks if $\mathbf{C}'[i] = \mathbf{C}^{(1)}[i] + c\mathbf{C}^{(2)}[i]$. (3) Finally, it forms the polynomial $f''_r \leftarrow f + cf'_r$ and checks if $f''_r(\mathbf{r}) = y_r + cy'_r$.

verifier $\mathcal{V}$. The simulator also has access to the sum K , oracle access to the polynomial f and works as follows:

- 1) The simulator samples the sumcheck polynomials $\{h_1(X), \dots, h_{\log n}(X)\}$ such that they produce an accepting sumcheck proof and sets $K' = h_1(0) + h_1(1)$. Then, it picks a matrix $\mathbf{R}_1$ uniformly at random such that $K' = \sum_{\mathbf{x}} f_r(\mathbf{x})$ and $f_r(\mathbf{r}) = y_r = b^{-1}(h_{\log n}(\mathbf{r}_{\log n}) - y)$. Finally, it samples $\mathbf{R}_2$ uniformly at random. Given these data, $\mathcal{S}$ simulates **Construction 5** as follows:
 - 2) It first simulates the $\mathcal{F}_{rand}^{Enc}$ functionality, by sending $\mathbf{C}^{(1)}_j$ and $\mathbf{C}^{(2)}_j$ for all $j \in \mathcal{J}$ to the adversary. Let V_1 be the view of the adversary at that phase.
 - 3) It simulates $\mathcal{F}_{ip}$ by sending K' to the adversary.
 - 4) It simulates $\mathcal{F}_{sumcheck}$ by sending $h_1(X), \dots, h_{\log n}(X), y, y_r$ to the adversary.
 - 5) It simulates step 5 by first sending y'_r , then $\mathbf{C}'$ and, finally, the replies $\mathbf{q}^{(1)}, \mathbf{q}^{(2)} \in \mathbb{F}^l$ of $\mathbf{C}^{(1)}$ and $\mathbf{C}^{(2)}$ at the indexes $\mathcal{I}$.

It is not hard to see that the view generated by the simulator is an accepting zero-knowledge proof for the sumcheck instance $K = \sum_{\mathbf{x}} f(\mathbf{x})$ sampled uniformly at random. Hence, we will focus on showing that the view obtained by running **Construction 5** in the real world follows a uniform distribution over all accepting proofs.

As a first step, we will show that for any such view

V , and polynomial f there exist bivariate polynomials $p_r, p'_r : \mathbb{F}^2 \rightarrow \mathbb{F}$ that produce the codewords $\mathbf{C}^{(1)}, \mathbf{C}^{(2)}$ and also the sumcheck polynomials. To achieve this, we first compute $h_r^{(i)}(X) \leftarrow b^{-1}(h^{(i)}(X) - h_f^{(i)}(X))$, where $h_f^{(i)}(X)$ the i -th sumcheck message of the instance $K = \sum_{\mathbf{x}} f(\mathbf{x})$. Then, we use them to compute the matrix $\mathbf{R}_1$ by solving a system of linear equations obtained by these conditions (similarly to what is done in [17]). Given that $\mathbf{C}^{(1)}$ is (t, l) -zero knowledge, from lemma 2 we know that there will always exist a polynomial p_r of correct degree that passes through $\mathbf{R}_1, \{\mathbf{C}^{(1)}_j\}_{j \in \mathcal{J}}$ and $\mathbf{q}^{(1)}$. We thus select a p_r , compute its codeword $\mathbf{C}^{(1)}$, and finally find $\mathbf{C}^{(2)} = \mathbf{C}' - \mathbf{C}^{(1)}$. From $\mathbf{C}^{(2)}$, we find p'_r .

What remains, is to show that the number of polynomials p_r, p'_r for any view is the same. Note that the number of different matrixes $\mathbf{R}_1$ we can find for a single view is $|\mathbb{F}|^{k \log n - \log n}$ since the system of linear equations described previously has full rank. Furthermore, for any such $\mathbf{R}_1$ we can interpolate the same number of polynomials p_r and subsequently p'_r . Consequently, the number of polynomials p_r, p'_r that can produce a single view is constant and the same with any other view. Hence, for any given p_r, p'_r and f , the probability we obtain a view V is constant. $\square$

5.1.3. Proof of Soundness.. First, we show that if the evaluation claim of the masked matrix is incorrect, that

is, $y_r \neq f_r(\mathbf{r})$, then the verifier accepts step 4 only with negligible probability. Initially, assume the event that $y'_r \neq f'_r(\mathbf{r})$ but $f_r(\mathbf{r}) + cy'_r = y_r + cy_r$. Due to the Schwartz-Zippel lemma, this can happen only with probability $\frac{1}{|\mathbb{F}|}$. Hence, $f_r(\mathbf{r}) + cy'_r \neq y_r + cy_r$. Next, assume that $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \frac{|\pi^{(1)}| \Delta}{3}$, where $\pi^{(1)}$ and $\pi^{(2)}$ the oracles of $\mathbf{R}_1, \mathbf{R}_2$ sent to the verifier. Then, by [2] we have that $\Delta(\pi^{(1)} + c\pi^{(2)}, \mathbf{C}) > \delta$ with all but $\frac{|\pi^{(1)}|}{|\mathbb{F}|}$ probability. So, the verifier would reject with probability $(1-\delta)^n < \text{negl}(\lambda)$. Otherwise, assume that $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) < \delta$, and let $\mathbf{C}^{(1)}, \mathbf{C}^{(2)}$ be the codewords closest to $\pi^{(1)}, \pi^{(2)}$. Observe that $\pi^{(1)}, \pi^{(2)}$ uniquely decode to $\mathbf{R}_1, \mathbf{R}_2$. Based on this, we consider two cases: (1) $\mathbf{C}^{(1)} + c\mathbf{C}^{(2)} = \mathbf{C}$ and (2) $\mathbf{C}^{(1)} + c\mathbf{C}^{(2)} \neq \mathbf{C}$. For (1), observe that it must hold that $\mathbf{R}' = \mathbf{R}_1 + b\mathbf{R}_2$ and thus, by our initial assumption, hence the verifier would reject with probability 1 since $f''(\mathbf{r}) \neq y_r + cy'_r$. For (2) we follow the same argument as in [18] (Section 4.1) to show that the chance of accepting all queries is at most $(1 - \frac{2}{3}\Delta)^n < \text{negl}(\lambda)$.

Having established that, we can now consider two cases. The first is that $K + bK' = \sum_{\mathbf{x}} (f(\mathbf{x}) + bf_r(\mathbf{x}))$. This however can happen only with probability $\frac{1}{|\mathbb{F}|}$. Consequently, we assume that $K + bK' \neq \sum_{\mathbf{x}} (f(\mathbf{x}) + bf_r(\mathbf{x}))$. Due to the soundness of the sumcheck protocol, the probability the verifier obtains an accepting proof is only $\frac{\log n}{|\mathbb{F}|}$. Consequently, with overwhelming probability it holds that $y_r \neq f_r(\mathbf{r})$. As discussed above, the verifier accepts it only with probability $(1 - \Delta/3)^l = \text{negl}(\lambda)$. By taking the union bound over all cases we end up with a soundness error of $\frac{\log n + 1}{|\mathbb{F}|} + (1 - \delta)^l$. $\square$

5.1.4. Proof of Round-by-Round (Knowledge) Soundness.. We first prove round-by-round soundness and then show how to modify the original proof in order to prove round-by-round (knowledge) soundness. Let $\mathbf{t}_0 = (K, \hat{f}, \pi^{(1)}, \pi^{(2)})$ be the initial state (where $\hat{f}$ is the oracle of f). We say that it is doomed if (1) $K \neq \sum_{\mathbf{x}} f(\mathbf{x})$ and $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) \leq \delta$ or (2) $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$.

Next, let $\mathbf{t}_1 = (\mathbf{t}_0, K', b)$. This state is doomed if (1) $K + bK' \neq \sum_{\mathbf{x}} (f(\mathbf{x}) + bf_r(\mathbf{x}))$ and $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) \leq \delta$ or (2) $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$. Observe that the probability ϵ_1 that $\mathbf{t}_0 \in \mathcal{D}$, where $\mathcal{D}$ the doomed set, but $\mathbf{t}_1 \notin \mathcal{D}$. We will show that this happens only with probability $\frac{1}{|\mathbb{F}|}$. Assume that $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$. Then the prover moves to a doomed state with probability 1. So we consider the case $K \neq \sum_{\mathbf{x}} f(\mathbf{x})$ and $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) \leq \delta$. Let f_r be the polynomial obtained by $\pi^{(1)}$. The prover can move to a non-doomed state if $K + bK' = \sum_{\mathbf{x}} (f(\mathbf{x}) + bf_r(\mathbf{x}))$. This however happens only with $\frac{1}{|\mathbb{F}|}$ probability.

Let $\mathbf{t}_j$ for $j \in \{2, 2 + \log n\}$ be the states obtained by the sumcheck protocol. We note that if $\mathbf{t}_j$ is doomed then $\mathbf{t}_{j+1}$ will be non-doomed with probability $\frac{1}{|\mathbb{F}|}$. To prove this we consider two cases. The first one is to start with a wrong sumcheck instance which implies a round-by-round soundness of $\frac{1}{|\mathbb{F}|}$ [12]. The other case is that

$\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \frac{\Delta}{3}$ where the round-by-round soundness is 0.

Let $\mathbf{t}_{3+\log n} = (\mathbf{t}_{2+\log n}, y, y_r, y'_r, c)$ be the state after the last sumcheck round. This state is doomed if (1) $h^{(\log n)}(r_{\log n}) \neq y + cy_r$, (2) $y \neq f(\mathbf{r})$ or $y_r \neq f_r(\mathbf{r})$ or (3) $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$. We will show that the probability that $\mathbf{t}_{2+\log n} \in \mathcal{D}$ but $\mathbf{t}_{3+\log n} \notin \mathcal{D}$ is $\frac{2 \log n}{|\mathbb{F}|}$. First, if $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$, then the prover remains in a doomed state with probability 1. To this point, the only way a prover can leave a doomed state is if $h^{(\log n)}(r_{\log n}) = y + cy_r$ and $y = f(\mathbf{r})$, $y_r = f_r(\mathbf{r})$. That can happen only with probability $\frac{2 \log n}{|\mathbb{F}|}$. Note that if $y \neq f(\mathbf{r})$, the prover can not leave the doomed state for the rest of the protocol hence verification will fail. So we assume that $y = f(\mathbf{r})$ and thus $y_r \neq f_r(\mathbf{r})$.

Let $\mathbf{t}_f = (\mathbf{t}_{3+\log n}, \mathbf{C}, \mathcal{I})$, where $\mathcal{I}$ the queries of the verifier over the oracles $\pi^{(1)}, \pi^{(2)}$. We want to show that if $\mathbf{t}_{3+\log n}$ is in a doomed state, that is, $y_r \neq f_r(\mathbf{r})$ or $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$ then $\mathbf{t}_f$ goes to a non-doomed state only with probability $(1 - \delta)^l$. This comes directly by the standard soundness proof of our scheme.

In summary, the round-by-round complexity of **Construction 5** is $\epsilon_{rbr} = \max(\epsilon_1, \dots, \epsilon_{3+\log n}, \epsilon_f) = \max(\frac{2 \log n}{|\mathbb{F}|}, (1 - \frac{\Delta}{3})^l)$.

Proving Round-by-Round Knowledge Soundness. Our proof is similar to that of proving round-by-round knowledge soundness of our coIOPP protocol. Here, we will provide a proof sketch on why $\kappa_{rbr} = \epsilon_{rbr}$. First, assume that $\mathbf{t}_f$ is not doomed but $\mathbf{t}_{3+\log n} \in \mathcal{D}$ with probability larger than ϵ_{rbr} . This means that (1) $y_r = f_r(\mathbf{r})$ (2) $y = f(\mathbf{r})$ and that the verifier passes all checks. Since $\mathbf{t}_{3+\log n} \in \mathcal{D}$, it must hold that $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) > \delta$. However the verifier passes the checks with probability larger than ϵ_{rbr} which indicates a contradiction. Consequently, $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) < \delta$ and $\mathbf{t}_{3+\log n} \notin \mathcal{D}$.

Next, assume that $\mathbf{t}_{3+\log n} \notin \mathcal{D}$ but $\mathbf{t}_{2+\log n} \in \mathcal{D}$ with probability larger than ϵ_{rbr} . Because $\mathbf{t}_{3+\log n} \notin \mathcal{D}$, it must hold that: (1) $h^{(\log n)}(r_{\log n}) = y + cy_r$, (2) $y = f(\mathbf{r})$ and $y_r = f_r(\mathbf{r})$ and (3) $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) \leq \delta$. Since $\mathbf{t}_{2+\log n} \in \mathcal{D}$, it must also hold that $h^{(\log n)}(X) \neq f'(\mathbf{r}, X) + cf'_r(\mathbf{r}, X)$ or $f'(\mathbf{x}) \neq f(\mathbf{x})$ or $f'_r(\mathbf{x}) \neq f_r(\mathbf{x})$. Based on this, and from the Schwartz-Zippel lemma, for any $r_{\log n}$ we move from $\mathbf{t}_{1+\log n} \in \mathcal{D}$ to $\mathbf{t}_{2+\log n} \notin \mathcal{D}$ only with probability of at most $\frac{\log n}{|\mathbb{F}|}$ which is smaller than ϵ_{rbr} , leading to a contradiction. We follow a similar argument for the rest of the sumcheck rounds.

Finally, assume that $\mathbf{t}_0 \in \mathcal{D}$ but $\mathbf{t}_1 \in \mathcal{D}$ with probability larger than ϵ_{rbr} . By definition, it must hold that $\Delta(\{\pi^{(1)}, \pi^{(2)}\}, RS^{\otimes 2}) \leq \delta$ and $K + bK' = \sum_{\mathbf{x}} (f'(\mathbf{x}) + bf_r(\mathbf{x}))$. Since, by assumption, $\mathbf{t}_0$ it must hold that $K \neq \sum_{\mathbf{x}} f(\mathbf{x})$ or $K' \neq \sum_{\mathbf{x}} f_r(\mathbf{x})$. By the Schwartz-Zippel lemma, the probability that $\mathbf{t}_1 \notin \mathcal{D}$ is at most $\frac{1}{|\mathbb{F}|} < \epsilon_{rbr}$ which leads to contradiction. So, $\mathbf{t}_0 \notin \mathcal{D}$ which concludes the proof. $\square$

5.2. Proof of Theorem 5.

Theorem 5. *The above protocol is a coPIOP for the relation $\mathcal{R}_C$ with perfect completeness, honest-verifier and prover t -zero knowledge with security threshold of $t \leq \frac{N}{2} - k + 1$, soundness error of $\epsilon = (3 \log m + 2 \log n + 10)/|\mathbb{F}| + 4(1 - \delta)^l$ and round-by-round (knowledge) soundness error of $\max(2 \log n/|\mathbb{F}|, 2 \log m/|\mathbb{F}|, (1 - \delta)^l)$. The non-interactive protocol resulted by the replacing the polynomial oracles of our PIOP with the coIOPP for $\mathcal{R}'_{MLE}$ and applying the collaborative BCS transformation is a coSNARK with $\mathcal{O}(\frac{|C|}{N} \log |C|)$ time complexity per-prover, $\mathcal{O}(\log^2 |C|)$ proof size and verification time, $\mathcal{O}(|C|)$ total communication complexity and $\mathcal{O}(\log |C|)$ communication rounds among provers.*

For context, we first give a brief formal description of our coPIOP protocol. We assume that all provers have Shamir's packed secret shares of $\mathbf{w}, \mathbf{u}_L, \mathbf{u}_R, \mathbf{u}_O$. In addition, we also assume that the provers have packed secret shares of the vectors $\mathbf{w}_r = (rnd, 0) \in \mathbb{F}^n$, $\mathbf{r}_1, \mathbf{r}_2 \in \mathbb{F}^2$ and $\mathbf{r}_3 \in \mathbb{F}^3$. The last vectors can be computed in a pre-processing step and they are used to mask the intermediate evaluation claims produced by the Spartan PIOP. Finally, we assume that the verifier holds as the instance the original instance $\mathbf{x}$ of $\mathcal{R}_C$, and the polynomial oracles of $f_A, f_B, f_O, f_{\mathbf{w}^*} : \mathbb{F}^{\log m+1} \rightarrow \mathbb{F}$, where $\mathbf{w}^* = (\mathbf{w}, \mathbf{w}_r)$. **Construction 6** gives a detailed description of the coPIOP.

The main difference with the protocol described in the main body is that the vectors original vectors $\mathbf{w}, \mathbf{u}_L, \mathbf{u}_R, \mathbf{u}_O$ and thus their multi-linear extensions are masked by appending the random vectors $\mathbf{w}_r, \mathbf{r}_1, \mathbf{r}_2$ and $\mathbf{r}_3$. Because of this modification, the original protocol changes as follows. First the provers send the oracles of $f_1, f_2, f_3 : \mathbb{F}^{\log m} \rightarrow \mathbb{F}$, which are the MLEs of $(\mathbf{r}_1, \mathbf{0}), (\mathbf{r}_2, \mathbf{0}), (\mathbf{r}_3, \mathbf{0}) \in \mathbb{F}^m$. Next, the verifier samples a random point $\mathbf{r} \in \mathbb{F}^{\log m}$ and sends it to the provers (step 2), which invoke the $\mathcal{F}_{ip}$ functionality to compute $K' = \sum_{i \in [3]} \beta(\mathbf{i}, \mathbf{r})(\mathbf{r}_1[i]\mathbf{r}_2[i] - \mathbf{r}_3[i])$. This step must be necessary since $\mathbf{r}_1 \circ \mathbf{r}_2 - \mathbf{r}_3$ is usually non-zero. Finally, the verifier selects another random point $\mathbf{r}' \in \mathbb{F}$ (step 3) and all parties interact over the sumcheck instance with input the masked polynomials f_L, f_R, f_O .

Likewise, we adapt the second sumcheck. For this, observe that $y_L = f_L(\mathbf{r}'_l, r_r) = (1 - r_r)f'_L(\mathbf{r}'_l) + r_r f_1(\mathbf{r}'_l)$, where f'_L is the MLE of $\mathbf{A}\mathbf{w}$. Using these “partial” evaluations, we expand them into sumcheck instances to form the final instance that is proven in step 7.

This is done by “expanding” the masked evaluation $y_L = f_L(\mathbf{r}'_l, r_r)$ (and y_R, y_O respectively) can be re-written as:

$$f_L(\mathbf{r}'_l, r_r) = \sum_{\mathbf{x} \in \{0,1\}^{\log n}} f_A(\mathbf{r}_l, \mathbf{x}) f_{\mathbf{w}}(\mathbf{x}) + r_r \sum_{\mathbf{x} \in [2]} \beta(\mathbf{x}, \mathbf{r}_l) f_1(\mathbf{x}) = \sum_{\mathbf{x} \in \{0,1\}^{\log n+1}} f_A(\mathbf{r}_l)$$

Lemma 9. *The coPIOP protocol of Construction 5 is t -zero knowledge with $t \leq \frac{N}{2} - k + 1 = (\frac{1}{2} - \epsilon)N + 1$, where $\epsilon \in (0, \frac{1}{2})$.*

Proof. We first construct a simulator $\mathcal{S}$ that has access to an adversary that controls t corrupted provers as well as the verifier. The simulator also has oracle access to the multi-linear extension of $\mathbf{w}$ and works as follows:

- 1) Sets $\mathbf{w} = \mathbf{0}$, samples uniformly at random the values $K', y_L, y_R, y_O, y_w, y_1, y_2, y_3$. Sets $rnd \leftarrow y_w \cdot \beta(\mathbf{n}, \mathbf{r}'')^{-1}$ and finds $\mathbf{r}_1, \mathbf{r}_2$ and $\mathbf{r}_3$ such that they satisfy the following constraints:
 - $K'(r')^{-1} = \beta(\mathbf{0}, \mathbf{r})(r_{31} - r_{11}r_{21}) + \beta(\mathbf{1}, \mathbf{r})(r_{32} - r_{12}r_{22}) + \beta(\mathbf{2}, \mathbf{r})r_{33}$.
 - $y_L \cdot \mathbf{r}'[\log m + 1]^{-1} = \beta(\mathbf{0}, \mathbf{r}'_l)r_{11} + \beta(\mathbf{1}, \mathbf{r}'_l)r_{12}$, where $\mathbf{r}_l$ has the $\log m$ least significant values.
 - $y_R \cdot \mathbf{r}'[\log m + 1]^{-1} = \beta(\mathbf{0}, \mathbf{r}'_l)r_{21} + \beta(\mathbf{1}, \mathbf{r}'_l)r_{22}$.
 - $y_O \cdot \mathbf{r}'[\log m + 1]^{-1} = \beta(\mathbf{0}, \mathbf{r}'_l)r_{31} + \beta(\mathbf{1}, \mathbf{r}'_l)r_{32} + \beta(\mathbf{2}, \mathbf{r}'_l)r_{33}$.
 - $y_M = \beta(\mathbf{0}, \mathbf{r}'')r_{M1} + \beta(\mathbf{1}, \mathbf{r}'')r_{M2} + \beta(\mathbf{2}, \mathbf{r}'')r_{M3}$, for $M \in [3]$. Note that for $M \in [2]$, $r_{M3} = 0$.
- 2) Note that if this system is solvable, then it has a unique solution. In addition, for any randomly selected $\mathbf{r}, \mathbf{r}', \mathbf{r}''$, the probability that the above system does not have a solution is negligible (inversely proportional to $|\mathbb{F}|$) [9], [18]. If the system is not solvable, the simulator aborts.
- 3) Sends the oracles of the MLEs of $(\mathbf{0}, \mathbf{w}_r)$ and $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3$ to the adversary.
- 4) Simulates the $\mathcal{F}_{ip}$ functionality by sending K' to the adversary.
- 5) Calls the simulator of our sumcheck protocol for the first sumcheck instance and obtains the view $V_{sumcheck}^{(1)}$.
- 6) Emulate the honest verifiers at step 5.
- 7) Calls the simulator of our sumcheck protocol for the second sumcheck instance and obtains the view $V_{sumcheck}^{(2)}$.

Let $V = \{V_1 = (K', y_L, y_R, y_O, y_w, y_1, y_2, y_3), V_{sumcheck}^{(1)}, V_{sumcheck}^{(2)}\}$ be the view obtained by simulator. Here we exclude the oracles since by default they are indistinguishable. Furthermore, we exclude y_A, y_B, y_C since they are publicly known. With that in mind, we want to show that V has the same distribution with the one obtained in the real world, namely, $V^* = \{V_1^* = (K'^*, y_L^*, y_R^*, y_O^*, y_w^*, y_1^*, y_2^*, y_3^*), V_{sumcheck}^{*(1)}, V_{sumcheck}^{*(2)}\}$. By the t -zero knowledge property **Construction 5**, we get that $V_{sumcheck}^{*(1)} \equiv V_{sumcheck}^{(1)}$ and $V_{sumcheck}^{*(2)} \equiv V_{sumcheck}^{(2)}$. We thus focus on showing that the rest of the elements are indistinguishable. Note that V_1 follows a uniform distribution over $\mathbb{F}^8$. So, we need to show that V_1^* also follows a uniform distribution. We will show that for any fixed $\mathbf{w}$, there exist $\mathbf{w}_r, \mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3$ that generate an accepting view V_1^* . Let E_1 be the event that the constraint system of V_1^* as described in the the simulator algorithm is solvable. Then there exist unique $rnd, \mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3$ that produce V_1^* . Consequently, conditioned E_1 , V_1^* is uniformly distributed. In other words:

$$\Pr[V \leftarrow \text{View}_{\mathcal{A}}^S(\mathbf{x}, \mathbf{w}_A) | E_1] \equiv \Pr[V^* \leftarrow \text{View}_{\mathcal{A}}^R(\mathbf{x}, \mathbf{w}) | E_1]$$

In addition, observe that from [9] the probability $\neg E$ happens is only when the expansions $\exp(\mathbf{r}), \exp(\mathbf{r}'), \exp(\mathbf{r}'')$

Construction 6: Our coPIOP. $\mathcal{P}_i$'s inputs. Each prover holds the RICS matrixes $(\mathbf{A}, \mathbf{B}, \mathbf{C})$, and the packed secret shares of (1) the witness $\mathbf{w}^* = (\mathbf{w}, \mathbf{w}_r = (rnd, \mathbf{0})) \in \mathbb{F}^{2n}$, where $rnd \in \mathbb{F}$ and (2) $\mathbf{u}_L = (\mathbf{A}\mathbf{w}^T, (\mathbf{r}_1, \mathbf{0}))$, $\mathbf{u}_R = (\mathbf{B}\mathbf{w}^T, (\mathbf{r}_2, \mathbf{0}))$, $\mathbf{u}_O = (\mathbf{C}\mathbf{w}^T, (\mathbf{r}_3, \mathbf{0})) \in \mathbb{F}^{2m}$, where $\mathbf{r}_1, \mathbf{r}_2 \in \mathbb{F}^2$ and $\mathbf{r}_3 \in \mathbb{F}^3$. Note that the secret shares of the random vectors have been produced in a pre-processing phase.

$\mathcal{V}$'s inputs. The verifier holds the polynomial oracles of the multi-linear extensions of the RICS matrixes $f_A, f_B, f_C : \mathbb{F}^{\log m + \log n} \rightarrow \mathbb{F}$.

Protocol Π^{coPIOP} . Let $f_L, f_R, f_O : \mathbb{F}^{\log m + 1} \rightarrow \mathbb{F}$ be the multi-linear extensions of $\mathbf{u}_L, \mathbf{u}_R, \mathbf{u}_O$. Each prover works as follows:

- 1) $\mathcal{P}_1, \dots, \mathcal{P}_N$: Send the oracles of $f_{\mathbf{w}^*} : \mathbb{F}^{1+\log n}$, $f_1, f_2, f_3 : \mathbb{F}^{\log m} \rightarrow \mathbb{F}$ the MLEs of $\mathbf{w}^*$, $(\mathbf{r}_1, \mathbf{0})$, $(\mathbf{r}_2, \mathbf{0})$ and $(\mathbf{r}_3, \mathbf{0})$ respectively.
- 2) $\mathcal{V}$: Sample a random point $\mathbf{r} \in \mathbb{F}^{\log m}$ and send it to the provers.
- 3) $\mathcal{P}_1, \dots, \mathcal{P}_N$: All provers invoke the $\mathcal{F}_{ip}$ functionality to compute $K' = \sum_i \beta(\mathbf{i}, \mathbf{r})(\mathbf{r}_1[i]\mathbf{r}_2[i] - \mathbf{r}_3[i])$. Finally, $\mathcal{P}_1$ sends K' to $\mathcal{V}$ which replies with another random point $\mathbf{r}'$.
- 4) $\mathcal{P}_1, \dots, \mathcal{P}_N, \mathcal{V}$: Set $\mathbf{r} \leftarrow (\mathbf{r}, \mathbf{r}')$ and interact following the coSumcheck protocol of **Construction 5** for proving the instance:

$$r'K' = \sum_{\mathbf{x} \in \{0,1\}^{1+\log m}} \beta(\mathbf{r}, \mathbf{x}) (f_L(\mathbf{x})f_R(\mathbf{x}) - f_O(\mathbf{x}))$$

At the end of the protocol, parties end up with the evaluation claims y_L, y_R, y_O of f_L, f_R, f_O at $\mathbf{r}'$.

- 5) $\mathcal{V}$: Sends the random points $a, b, c \in \mathbb{F}$.
- 6) $\mathcal{P}_1, \dots, \mathcal{P}_N$: All provers collaborate to compute the packed secret shares of the vector $\mathbf{p}$, the MLE of which is the polynomial $f_p(\mathbf{x}) = af_A(\mathbf{r}', \mathbf{x}) + bf_B(\mathbf{r}', \mathbf{x}) + cf_C(\mathbf{r}', \mathbf{x})$. Then set $\mathbf{p}' = (\mathbf{p}, \mathbf{0}) \in \mathbb{F}^{2n}$. In addition, compute the secret shares of the vector $\mathbf{b} = [\beta((0, \mathbf{0}), \mathbf{r}'), \dots, \beta((0, \mathbf{n-1}), \mathbf{r}')] \in \mathbb{F}^n$. Set $\mathbf{b}' = (\mathbf{b}, \mathbf{0}) \in \mathbb{F}^{2n}$.
- 7) $\mathcal{P}_1, \dots, \mathcal{P}_N, \mathcal{V}$: Interact following the coSumcheck protocol of **Construction 5** for proving the instance:

$$ay_L + by_R + cy_O = \sum_{\mathbf{x} \in \{0,1\}^{1+\log n}} (f_{\mathbf{p}'}(\mathbf{x})f_w(\mathbf{x}) + f_{\mathbf{b}'}(\mathbf{x})(af_1(\mathbf{x}) + bf_2(\mathbf{x}) + cf_3(\mathbf{x})))$$

At the end of the protocol, the verifier ends up with the evaluation claims $y_w, y_A, y_B, y_C, y_{b'}$ and y_1, y_2, y_3 of the polynomials $f_{w'}, f_A, f_B, f_C, f_{b'}, f_1, f_2, f_3$ at $\mathbf{r}''$.

- 8) $\mathcal{V}$: Validate $y_{b'}$ locally and for the rest of the polynomials use their respective oracles.

(with, $\exp(\mathbf{r}) = [\beta(\mathbf{0}, \mathbf{r}), \dots, \beta(\mathbf{n}, \mathbf{r})]$) do not lead to a rank three system which happens only with probability $\frac{3}{|\mathbb{F}|}$. Consequently, $\Pr[V^* \leftarrow \text{View}_A^{\mathcal{R}}(\mathbf{x}, \mathbf{w}) | \neg E_1] = \frac{3}{|\mathbb{F}|}$ [9]. Thus, the statistical distance between the distributions $\Pr[V \leftarrow \text{View}_A^{\mathcal{S}}(\mathbf{x}, \mathbf{w}_A)]$ and $\Pr[V^* \leftarrow \text{View}_A^{\mathcal{R}}(\mathbf{x}, \mathbf{w})]$ is $\frac{3}{|\mathbb{F}|}$.

Security Threshold. All sumchecks in our protocol involve two secret shared polynomials but one of them also involves a public polynomial that is multi-linear extension of a vector with a tensor-like structure. We know that the security threshold of these sumchecks is $t \leq \frac{N}{2} - k + 1$ [15]. Finally, using the protocols of [16], [15] the security threshold of $\mathcal{F}_{ip}$ for two secret matrixes is also $t \leq \frac{N}{2} - k + 1$. Hence, the overall security threshold of our construction is $t \leq \frac{N}{2} - k + 1$. $\square$ $\square$

Lemma 10. *Construction 6 is a coIOPP with soundness error $\epsilon = \frac{3 \log m + 2 \log n + 10}{|\mathbb{F}|} + 4(1 - \delta)^l$.*

Proof. Let E_1 be the event that $K' \neq \sum_i \beta(\mathbf{i}, \mathbf{r})(\mathbf{r}_1[i]\mathbf{r}_2[i] - \mathbf{r}_3[i])$ or there exists $\mathbf{u}_L[i]\mathbf{u}_R[i] \neq \mathbf{u}_O[i]$ for $i \in [m]$. By the Schwartz-Zippel Lemma, for any $\mathbf{r}$, the probability that the first sumcheck instance is $\frac{1+\log m}{|\mathbb{F}|}$. Next, let E_2 be the even

that the instance does not hold, but at the end of the protocol the verifier ends up with an accepting proof and correct evaluation claims. Due to the soundness of **Construction 5**, this can happen only with probability $\frac{2 \log m + 3}{|\mathbb{F}|} + 2(1 - \delta)^l$. Now, assume E_3 be the event that there exists an invalid y_L, y_R, y_O but the sumcheck instance is valid. This can happen only with probability $\frac{3}{|\mathbb{F}|}$ due to the Schwartz-Zippel Lemma. Finally, let E_4 be the event that the sumcheck instance is invalid but the provers generate an accepting proof and that the evaluation claims are valid. Due to the soundness of **Construction 5** this can happen only with probability $\frac{2 \log n + 3}{|\mathbb{F}|} + 2(1 - \delta)^l$.

By taking the union bound over these events, we get a soundness error of $\epsilon = \frac{3 \log m + 2 \log n + 10}{|\mathbb{F}|} + 4(1 - \delta)^l$. $\square$ $\square$

Lemma 11. *Construction 6 has round-by-round (knowledge) soundness errors $\kappa_{rbr} = \epsilon_{rbr} = \max(\frac{2 \log n}{|\mathbb{F}|}, \frac{2 \log m}{|\mathbb{F}|}, (1 - \delta)^l)$.*

Proof. Let $\mathbf{t}_0 = (\pi_{pr}, f_{\mathbf{w}^*})$ be the initial state, where π_{pr} contains the oracles of the mask codewords computed in the pre-processing phase of the sumcheck protocol. We say that $\mathbf{t}_0$ is doomed if (1) $\exists i \in [m]$, such that $\mathbf{u}_L[i]\mathbf{u}_R[i] \neq$

$\mathbf{u}_O[i]$, or (2) if there exists a masking codeword such that $\Delta(\pi, \mathbf{C}) > \delta$. Let $\mathbf{t}_1 = (\mathbf{t}_0, f_1, f_2, f_3, \mathbf{r})$. If $\mathbf{t}_0 \in \mathcal{D}$, then, the probability ϵ_1 $\mathbf{t}_1$ leaves a doomed state is at most $\frac{\log m}{|\mathbb{F}|}$. To prove this we consider two cases: (a) $\mathbf{u}_L \circ \mathbf{u}_R \neq \mathbf{u}_O$, so the probability to leave a doomed state is if and only if $0 = \sum_{\mathbf{x}} \beta(\mathbf{r}, \mathbf{x})(f_L(\mathbf{x})f_R(\mathbf{x}) - f_O(\mathbf{x}))$, which due to the Schartz-Zippel Lemma happens only with probability $\frac{\log m}{|\mathbb{F}|}$. (b) $\Delta(\pi, \mathbf{C}) > \delta$ holds, so the prover stays in a doomed state with probability 1.

Next, assume that $\mathbf{t}_1$ is in a doomed state. Then, with probability $\epsilon_2 = \frac{1}{|\mathbb{F}|}$, $\mathbf{t}_2 = (\mathbf{t}_1, K', r')$ will be in a doomed state. We consider two cases: (a) $0 \neq \sum_{\mathbf{x}} \beta(\mathbf{r}, \mathbf{x})(f_L(\mathbf{x})f_R(\mathbf{x}) - f_O(\mathbf{x}))$, so due to the Schwartz-Zippel lemma, the sumcheck instance becomes accepting only with probability $\frac{1}{|\mathbb{F}|}$. (b) $\Delta(\pi, \mathbf{C}) > \delta$ holds so the prover stays in a doomed state with probability 1.

Let $\mathbf{t}_3 = (\mathbf{t}_2, \mathbf{t}_{smc1})$ be the state after the sumcheck protocol of step 4. If $\mathbf{t}_2$ is in a doomed state, then, from Theorem 4, with probability $\epsilon_3 = \max(\frac{2 \log m}{|\mathbb{F}|}, (1 - \delta)^l)$, $\mathbf{t}_3$ will remain in a doomed state. For the rest of this proof we omit the case where the sumcheck verifier fails since the prover will permanently stay in doomed state.

Assume that $\mathbf{t}_3$, then by the Schwartz-Zippel lemma, $\mathbf{t}_4 = (\mathbf{t}_3, a, b, c)$ will stay in a doomed state with probability $\epsilon_4 = \frac{3}{|\mathbb{F}|}$. Finally, let $\mathbf{t}_f = (\mathbf{t}_4, \mathbf{t}_{smc2})$ be the final transcript. By the round-by-round soundness of our coSum-check, the probability that $\mathbf{t}_4$ is doomed and $\mathbf{t}_f$ is not, is $\epsilon_5 = \max(\frac{2 \log n}{|\mathbb{F}|}, (1 - \delta)^l)$.

To sum up, the round-by-round soundness error of **Construction 6** is $\epsilon_{rbr} = \max(\epsilon_1, \epsilon_2, \epsilon_3, \epsilon_4, \epsilon_5) = \max(\frac{2 \log n}{|\mathbb{F}|}, \frac{2 \log m}{|\mathbb{F}|}, (1 - \delta)^l)$. Similarly to the proof of Theorems 4- 3 we prove round-by-round knowledge soundness. $\square$

References

- [1] N. Ron-Zewi and M. Weiss, “Zero-knowledge iops approaching witness length,” in *CRYPTO*, 2024, pp. 105–137.
- [2] S. Ames, C. Hazay, Y. Ishai, and M. Venkatasubramanian, “Ligero: Lightweight sublinear arguments without a trusted setup,” in *ACM CCS*, 2017, pp. 2087–2104.
- [3] E. Ben-Sasson, A. Chiesa, and N. Spooner, “Interactive oracle proofs,” in *TCC*, 2016, pp. 31–60.
- [4] A. Chiesa, P. Manohar, and N. Spooner, “Succinct arguments in the quantum random oracle model,” in *TCC*, 2019, pp. 1–29.
- [5] A. Chiesa, D. Ojha, and N. Spooner, “Fractal: Post-quantum and transparent recursive proofs from holography,” in *EUROCRYPT*, 2020, pp. 769–793.
- [6] I. Damgård, Y. Ishai, and M. Krøigaard, “Perfectly secure multiparty computation and the computational overhead of cryptography,” in *EUROCRYPT*, 2010, pp. 445–465.
- [7] M. K. Franklin and M. Yung, “Communication complexity of secure computation (extended abstract),” in *ACM STOC*, 1992, pp. 699–710.
- [8] E. Ben-Sasson, S. Kopparty, and S. Saraf, “Worst-case to average case reductions for the distance to a code,” in *CCC*, 2018, pp. 24:1–24:23.
- [9] H. Zeilberger, B. Chen, and B. Fisch, “Basefold: Efficient field-agnostic polynomial commitment schemes from foldable codes,” in *CRYPTO*, 2024, pp. 138–169.
- [10] J. Zhang, T. Xie, Y. Zhang, and D. Song, “Transparent polynomial delegation and its applications to zero knowledge proof,” in *IEEE SP*, 2020, pp. 859–876.
- [11] T. Liu, X. Xie, and Y. Zhang, “zkcn: Zero knowledge proofs for convolutional neural network predictions and accuracy,” in *CCS*, 2021, pp. 2968–2985.
- [12] R. Canetti, Y. Chen, J. Holmgren, A. Lombardi, G. N. Rothblum, and R. D. Rothblum, “Fiat-shamir from simpler assumptions,” *IACR Cryptol. ePrint Arch.*, 2018.
- [13] P. Valiant, “Incrementally verifiable computation or proofs of knowledge imply time/space efficiency,” in *TCC*, 2008, pp. 1–18.
- [14] A. R. Block, A. Garreta, J. Katz, J. Thaler, P. R. Tiwari, and M. Zajac, “Fiat-shamir security of FRI and related snarks,” in *ASIACRYPT*, 2023, pp. 3–40.
- [15] X. Liu, Z. Zhou, Y. Wang, Y. Pang, J. He, B. Zhang, X. Yang, and J. Zhang, “Scalable collaborative {zk-SNARK} and its application to fully distributed proof delegation,” in *(USENIX Security)*, 2025.
- [16] X. Liu, Z. Zhou, Y. Wang, B. Zhang, and X. Yang, “Scalable collaborative zk-snark: fully distributed proof generation and malicious security,” *Cryptology ePrint Archive*, 2024.
- [17] T. Xie, J. Zhang, Y. Zhang, C. Papamanthou, and D. Song, “Libra: Succinct zero-knowledge proofs with optimal prover computation,” in *CRYPTO*, 2019, pp. 733–764.
- [18] A. Golovnev, J. Lee, S. T. V. Setty, J. Thaler, and R. S. Wahby, “Brakedown: Linear-time and field-agnostic snarks for R1CS,” in *CRYPTO*, 2023, pp. 193–226.